

DISEÑO ARQUITECTÓNICO DE SOFTWARE

Diseño Arquitectónico usando el Modelo de Vistas C4

Sistema: TradeNova — Plataforma de Trading Financiero en Tiempo Real

Autor:

Alejandro De Mendoza

Institución Universitaria Politécnico Grancolombiano

Maestría en Arquitectura de Software

Diseño Arquitectónico de Software

Docente: Ing. John Rondón Suárez

Junio 2026

Tabla de Contenido

Tabla de Contenido	2
1. Propósito	3
2. Descripción del Contexto del Sistema.....	4
2.1 El Negocio: TradeNova.....	4
2.2 Problemática Arquitectónica	4
2.3 Solución Propuesta.....	4
3. Instalación y Configuración de Archi	5
3.1 Descarga de Archi	5
3.2 Proceso de Instalación	6
3.3 Primera Ejecución.....	7
3.4 Interfaz Principal y Configuración del Proyecto	8
4. Implementación del Modelo C4.....	9
4.1 Diagrama de Contexto (Nivel 1).....	9
Descripción	9
Proceso de construcción en Archi.....	9
Elementos del Diagrama.....	14
Diagrama Final.....	14
Análisis de SLAs, Dependencias y Constraints Regulatorios	15
Trade-offs Arquitectónicos del Contexto	16
4.2 Diagrama de Contenedores (Nivel 2)	16
Descripción	16
Proceso de construcción en Archi.....	17
Elementos del Diagrama.....	18
Diagrama Final.....	19
Estrategia de Deployment — Kubernetes y Multi-Region	19
Observabilidad — Logs, Tracing y Metrics.....	20
Manejo de Fallos Distribuidos — Resiliencia	20
4.3 Diagrama de Componentes (Nivel 3).....	21
Descripción	21
Proceso de construcción en Archi.....	21
Elementos del Diagrama.....	23
Diagrama Final.....	23
Pipeline Reactivo — Paralelismo, Backpressure y Retry Strategies	24
Seguridad Avanzada en el Motor de Órdenes	25
4.4 Diagrama de Código (Nivel 4).....	25

Descripción	25
Proceso de construcción en Archi.....	26
Elementos del Diagrama.....	27
Diagrama Final.....	27
Patrones de Diseño Aplicados	28
Manejo de Errores Estructurado	29
Consistencia de Datos — ACID vs. Eventual Consistency	29
6. Consideraciones de Arquitectura Global	30
6.1 Seguridad — Zero Trust Architecture	30
6.2 DevOps — CI/CD e Infraestructura como Código	31
6.3 Consistencia y Gestión de Eventos en Kafka	32
6.4 Estrategia de Recuperación ante Desastres	33
7. Exportación de los Diagramas	33
8. Conclusiones.....	35
9. Referencias Bibliográficas.....	36

1. Propósito

El propósito de este documento consiste en presentar el diseño arquitectónico de la plataforma financiera TradeNova mediante la aplicación del Modelo de Vistas C4 (Context, Containers, Components, Code). Este enfoque permite representar la arquitectura del sistema en diferentes niveles de abstracción, facilitando la comprensión de la solución tanto para perfiles técnicos como no técnicos involucrados en el proyecto.

El Modelo C4, desarrollado por Simon Brown, proporciona un lenguaje visual claro y estructurado para documentar arquitecturas de software modernas. Organiza la representación arquitectónica en cuatro niveles jerárquicos que van desde una vista general del sistema y su entorno, hasta el detalle de las clases y módulos de código que implementan funcionalidades específicas.

En el contexto de TradeNova, la aplicación del Modelo C4 cobra especial relevancia dado que la plataforma está siendo sometida a una transformación arquitectónica de gran envergadura: la migración desde una arquitectura monolítica tradicional hacia una arquitectura moderna basada en microservicios, procesamiento orientado a eventos y despliegue en infraestructura cloud-native. Esta transición implica decisiones arquitectónicas complejas relacionadas con la distribución de responsabilidades entre componentes, la comunicación entre servicios, la gestión de datos distribuidos y la garantía de atributos de calidad críticos como alta disponibilidad, ultra-baja latencia, escalabilidad automática y seguridad regulatoria.

Los diagramas C4 presentados en este informe han sido elaborados utilizando la herramienta Archi 5.9.0 con notación ArchiMate 3.2, permitiendo representar de manera formal y estandarizada la estructura interna del sistema, las relaciones con actores y sistemas externos, la organización de los contenedores tecnológicos y la descomposición en componentes funcionales.

2. Descripción del Contexto del Sistema

2.1 El Negocio: TradeNova

TradeNova es una plataforma emergente de corretaje financiero (retail brokerage) orientada a usuarios minoristas que desean operar en tiempo real con instrumentos financieros tales como acciones, ETFs y opciones bursátiles, a través de aplicaciones móviles y entornos web. La plataforma busca ofrecer una experiencia de trading rápida, segura y altamente disponible, permitiendo a los usuarios ejecutar operaciones financieras en mercados de alta volatilidad con acceso inmediato a información de mercado y procesamiento transaccional en tiempo real.

Inicialmente desarrollada bajo una arquitectura monolítica tradicional, TradeNova comenzó a presentar limitaciones críticas asociadas al crecimiento acelerado de usuarios y al incremento exponencial del volumen de transacciones procesadas. Durante eventos de alta volatilidad financiera —como aperturas bursátiles, anuncios de la Reserva Federal o fluctuaciones extremas de activos— la plataforma experimenta saturación de recursos, incremento de latencia, bloqueos en la interfaz de usuario y ejecución de órdenes con información desactualizada.

2.2 Problemática Arquitectónica

Problema	Impacto
Saturación en picos de demanda	Latencia elevada, órdenes rechazadas o ejecutadas incorrectamente
Escalabilidad limitada	Imposibilidad de crecer horizontalmente sin rediseñar todo el sistema
Despliegues riesgosos	Cualquier cambio afecta todo el sistema, generando ventanas de mantenimiento extensas
Acoplamiento alto	Dificultad para incorporar nuevos instrumentos financieros o módulos
Observabilidad deficiente	Diagnóstico lento de incidentes en producción

2.3 Solución Propuesta

Como respuesta a esta problemática, TradeNova plantea la migración hacia una arquitectura moderna basada en microservicios, eventos y servicios distribuidos desplegados sobre infraestructura cloud-native. Los objetivos principales de esta transformación son:

- Garantizar resiliencia operativa y alta disponibilidad (99.99% uptime)
- Lograr ultra-baja latencia en el procesamiento de órdenes financieras (< 10 ms)
- Implementar escalabilidad automática ante picos de demanda (escalamiento en < 60 segundos)
- Cumplir con requisitos regulatorios financieros estrictos mediante trazabilidad completa
- Permitir la evolución independiente de cada microservicio sin afectar el núcleo transaccional
- Integrar de manera eficiente con ecosistemas financieros externos mediante protocolo FIX

3. Instalación y Configuración de Archi

Para la elaboración de los diagramas C4 se utilizó Archi 5.9.0, la herramienta de modelado ArchiMate de referencia en la industria, distribuida de forma gratuita y de código abierto. A continuación se documenta el proceso completo de instalación y configuración de la herramienta.

3.1 Descarga de Archi

La herramienta Archi se descarga desde su sitio oficial en <https://www.archimatetool.com/download/>. Se seleccionó la versión para Windows Installer (Archi 5.9.0), compatible con Windows 10+ de 64 bits, que instala la aplicación en la carpeta Program Files y registra la asociación con archivos .archimate.

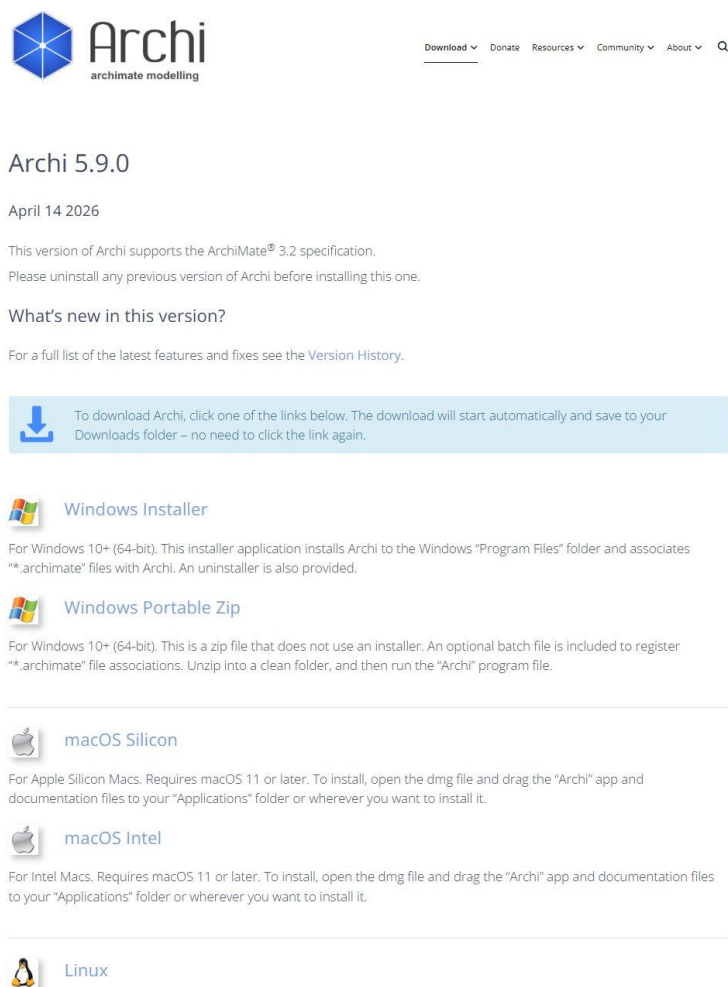


Figura 1. Página oficial de descarga de Archi 5.9.0 con opciones para Windows, macOS y Linux

3.2 Proceso de Instalación

El proceso de instalación se realizó mediante el asistente de Windows, siguiendo los pasos estándar de configuración de directorio de instalación, carpeta del menú de inicio y creación de acceso directo en el escritorio.

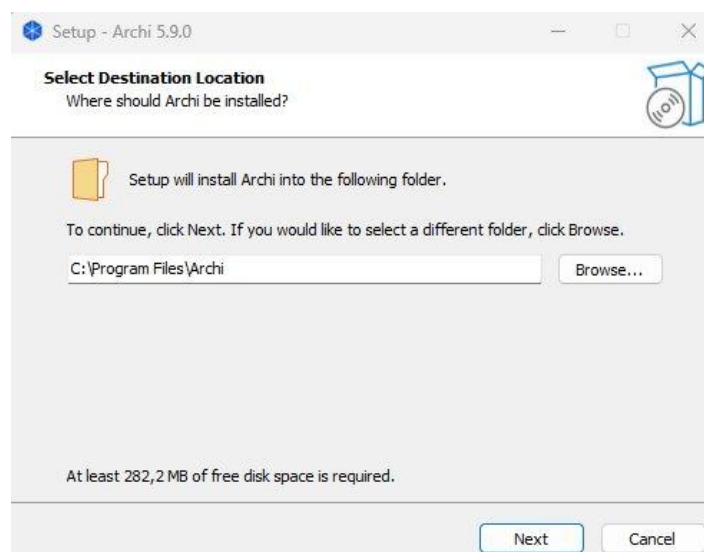
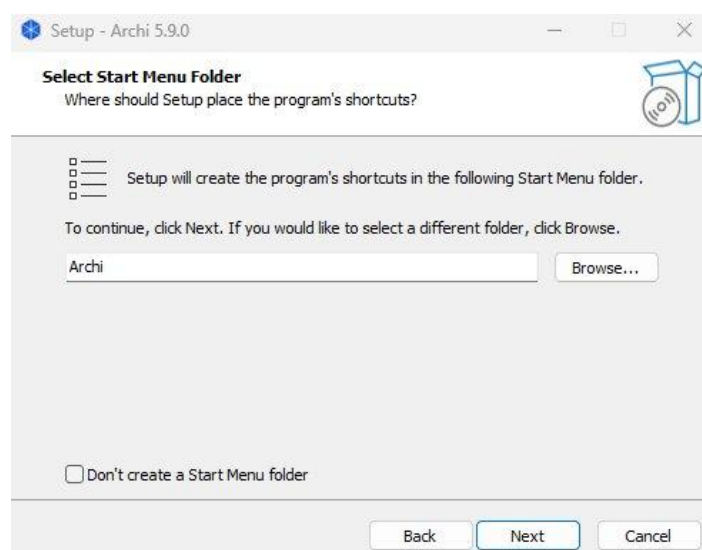
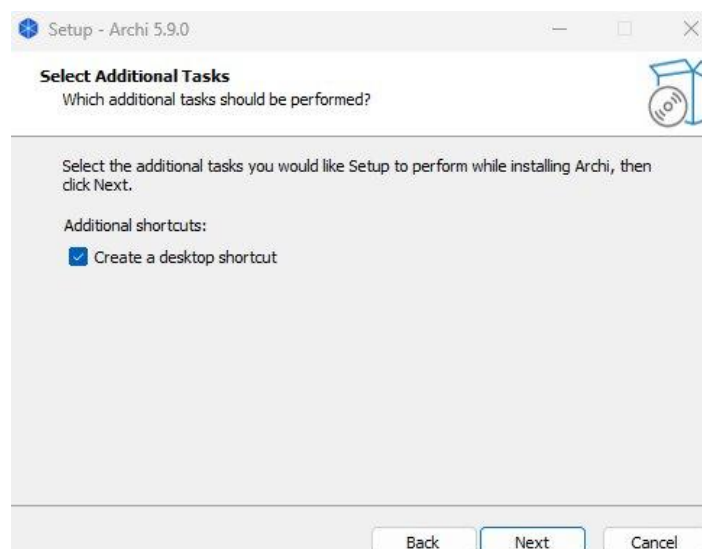


Figura 2. Selección del directorio de instalación — C:\Program Files\Archi*Figura 3. Configuración de la carpeta del menú de inicio**Figura 4. Selección de tareas adicionales — creación de acceso directo en el escritorio*

3.3 Primera Ejecución

Al ejecutar Archi por primera vez, la aplicación presenta una pantalla de bienvenida con las opciones principales: crear un nuevo modelo ArchiMate o acceder directamente a la aplicación. Se seleccionó la opción de crear un nuevo modelo para iniciar el proyecto TradeNova C4.

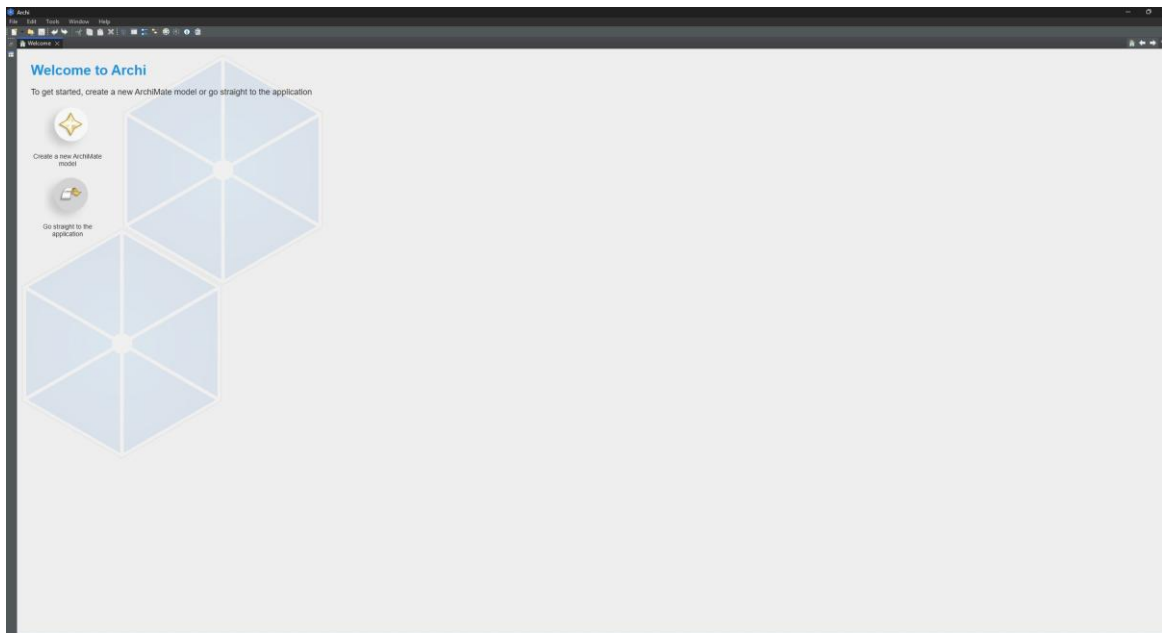


Figura 5. Pantalla de bienvenida de Archi 5.9.0 con opciones de inicio

3.4 Interfaz Principal y Configuración del Proyecto

La interfaz principal de Archi se organiza en cuatro paneles fundamentales: el panel Models (izquierda) con el árbol de elementos y vistas del proyecto, el Canvas central donde se construyen los diagramas, la Palette (derecha) con todos los elementos ArchiMate disponibles para arrastrar, y el panel Properties (inferior) con las propiedades del elemento seleccionado.

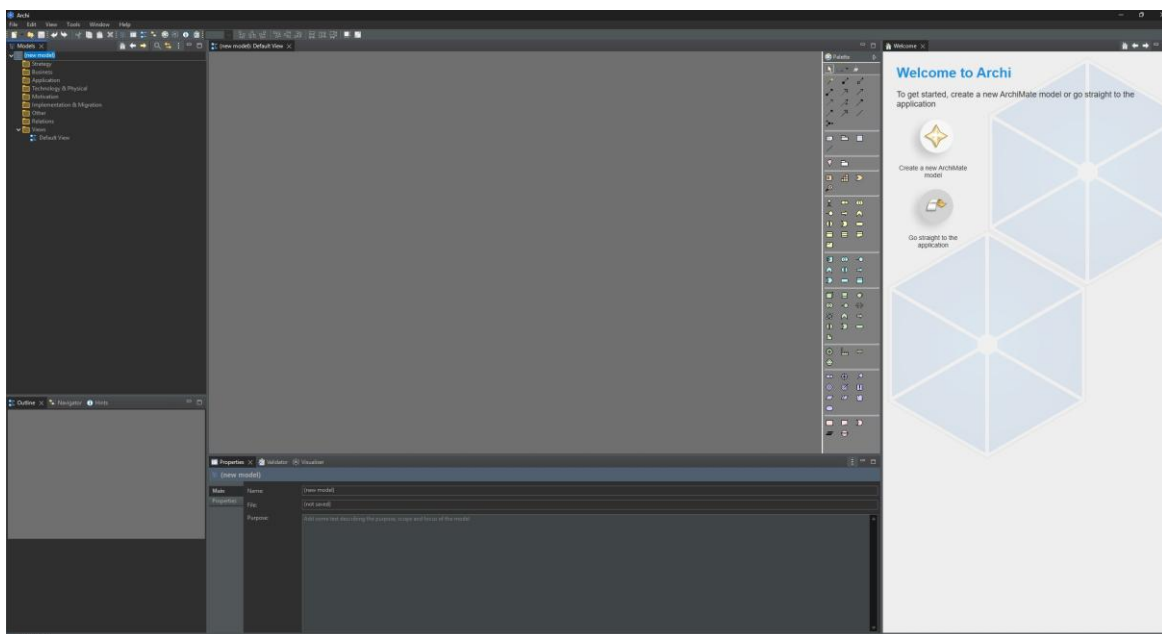


Figura 6. Interfaz principal de Archi con nuevo modelo creado y paneles identificados

Se configuró el modelo con el nombre TradeNova C4 en el campo Name del panel Properties, y el archivo fue guardado en la ruta del laboratorio: C:\Users\P1 3-

3\Documents\Estudio\Maestría Arquitectura Software\Diseño Arquitectonico Software\Laboratorio No. 2\TradeNova_C4.archimate.

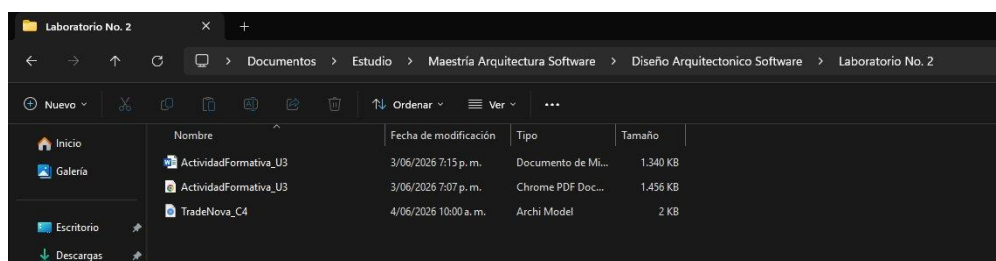


Figura 7. Archivo TradeNova_C4.archimate guardado en la carpeta del Laboratorio No. 2

4. Implementación del Modelo C4

El Modelo C4 organiza la documentación arquitectónica en cuatro niveles de abstracción que permiten comunicar la arquitectura del sistema de manera progresiva y clara. Cada nivel responde a una audiencia y propósito específico, comenzando desde una visión general del sistema hasta el detalle del código interno de sus componentes más críticos.

Nivel	Diagrama	Audiencia	Propósito
1	Contexto	Todos los stakeholders	Vista del sistema y su entorno
2	Contenedores	Arquitectos y líderes técnicos	Piezas tecnológicas del sistema
3	Componentes	Desarrolladores	Interior de un contenedor
4	Código	Desarrolladores	Clases y módulos internos

4.1 Diagrama de Contexto (Nivel 1)

Descripción

El Diagrama de Contexto representa el nivel más alto de abstracción del Modelo C4. Su objetivo principal consiste en mostrar TradeNova como una caja negra, identificando los actores humanos y sistemas externos que interactúan directamente con la plataforma. Este diagrama establece claramente las fronteras del sistema y sus puntos de integración con el entorno externo.

Proceso de construcción en Archi

Para construir el Diagrama de Contexto se creó una nueva vista ArchiMate denominada '01 - Diagrama de Contexto' haciendo clic derecho sobre Views en el

panel Models. A continuación se identificaron y configuraron los elementos de la paleta correspondientes a cada actor y sistema del entorno.

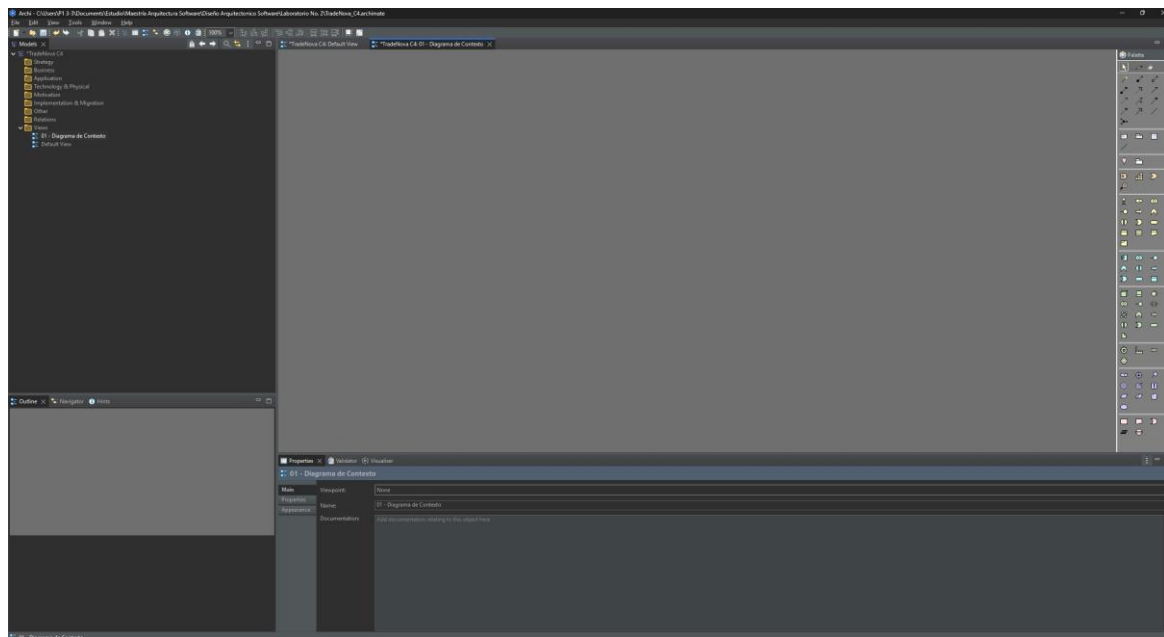


Figura 8. Vista 01 - Diagrama de Contexto creada en el panel Models de Archi

La Palette de Archi organiza los elementos ArchiMate por capas. Para este diagrama se utilizaron principalmente elementos de la capa Business (Business Actor — figura humana, color amarillo) y la capa Application (Application Component — rectángulo con dos cuadros en la esquina superior derecha, color celeste).

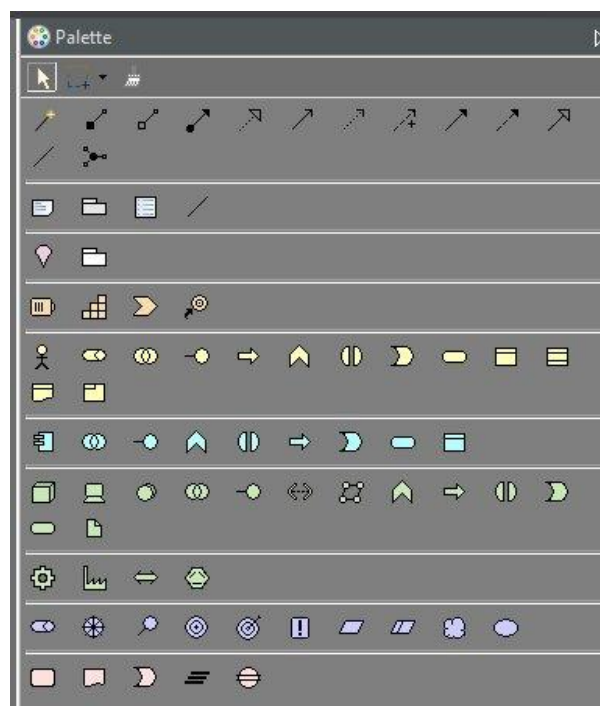


Figura 9. Palette de Archi con los elementos disponibles por capas

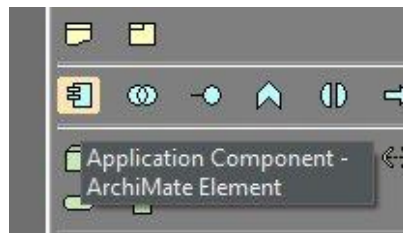


Figura 10. Identificación del elemento Application Component en la Palette

Se agregó el elemento central TradeNova como Application Component al canvas, seguido de los Business Actors (Usuario Trader y Regulador Financiero) y los sistemas externos (Broker Externo, Bolsa de Valores, Proveedor de Datos) también como Application Components.

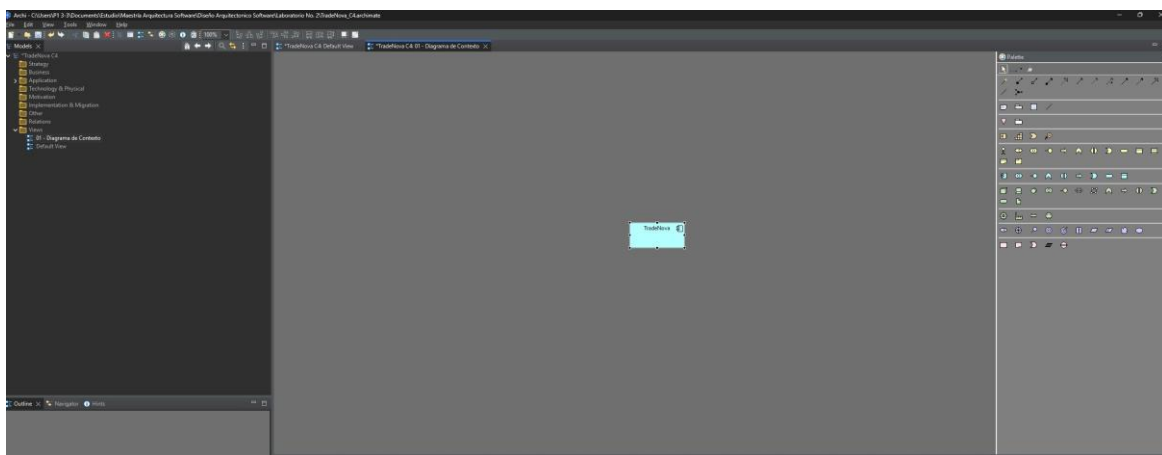


Figura 11. Elemento TradeNova (Application Component) agregado al canvas

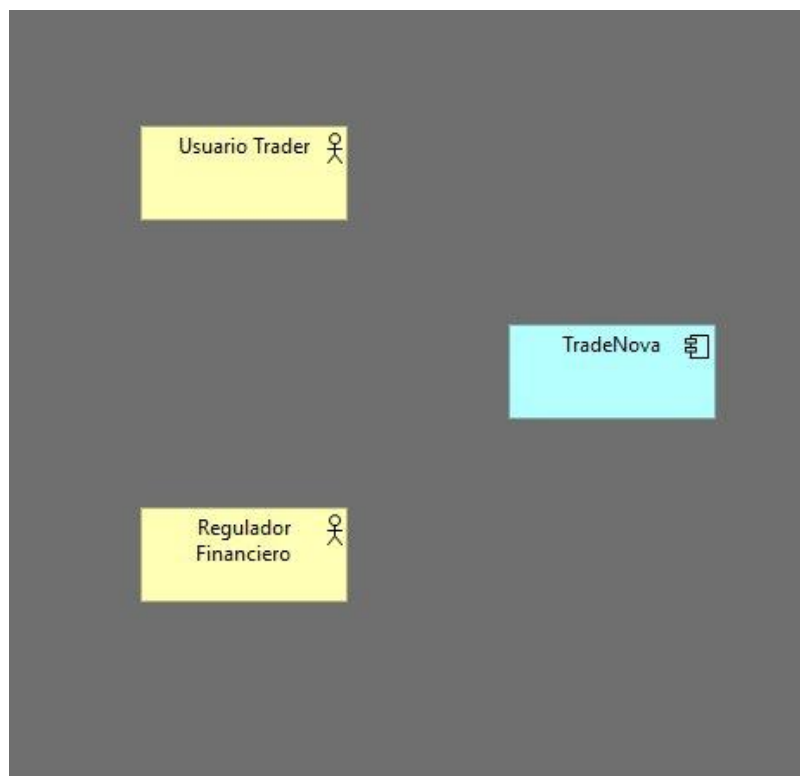
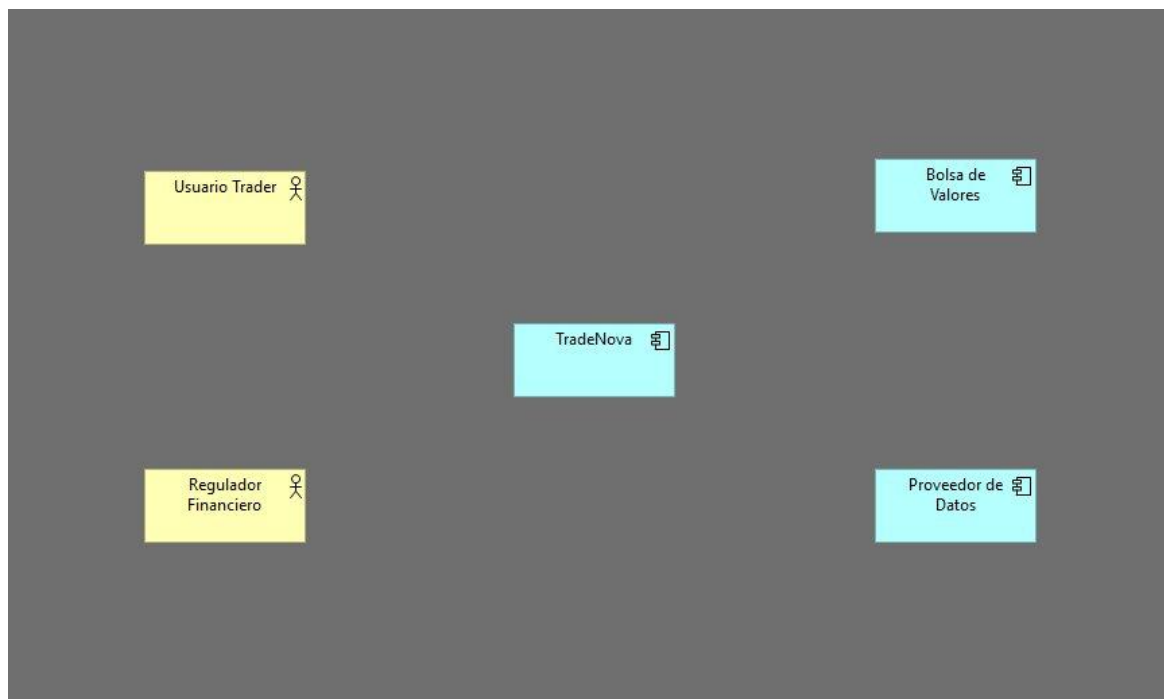
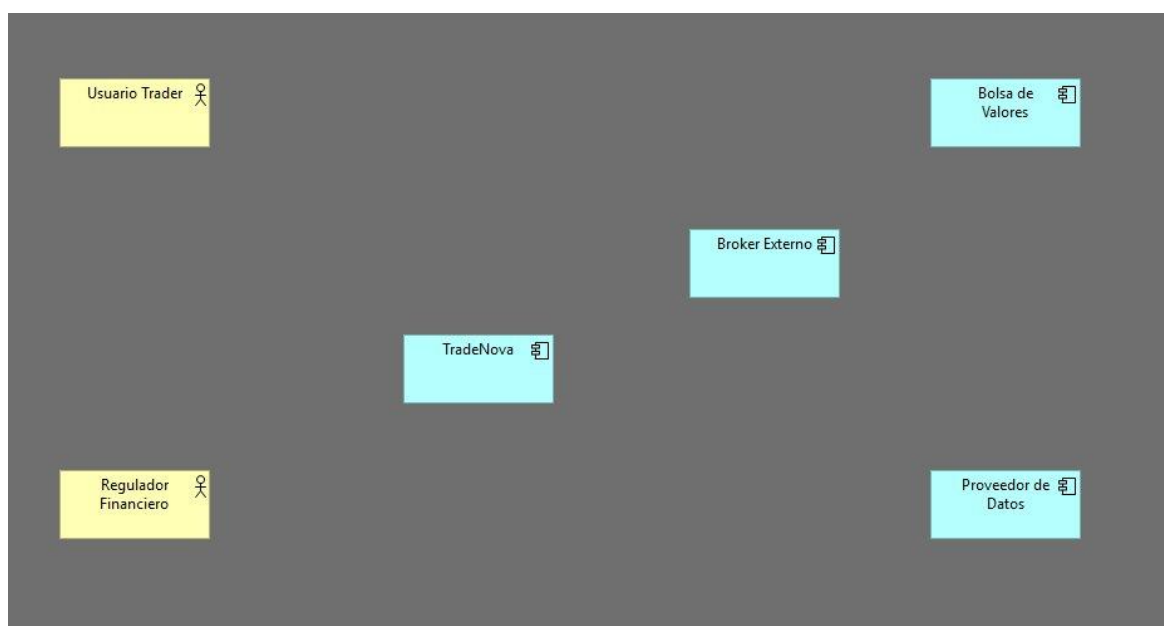


Figura 12. Business Actors agregados — Usuario Trader y Regulador Financiero (color amarillo)*Figura 13. Sistemas externos agregados — Bolsa de Valores y Proveedor de Datos**Figura 14. Diagrama con todos los elementos — incluyendo Broker Externo*

Para conectar los elementos se utilizaron relaciones de Association (línea sólida con flecha abierta), arrastrando desde el borde de cada elemento hasta el destino. Al completar las conexiones se agregaron etiquetas descriptivas haciendo doble clic sobre cada flecha.

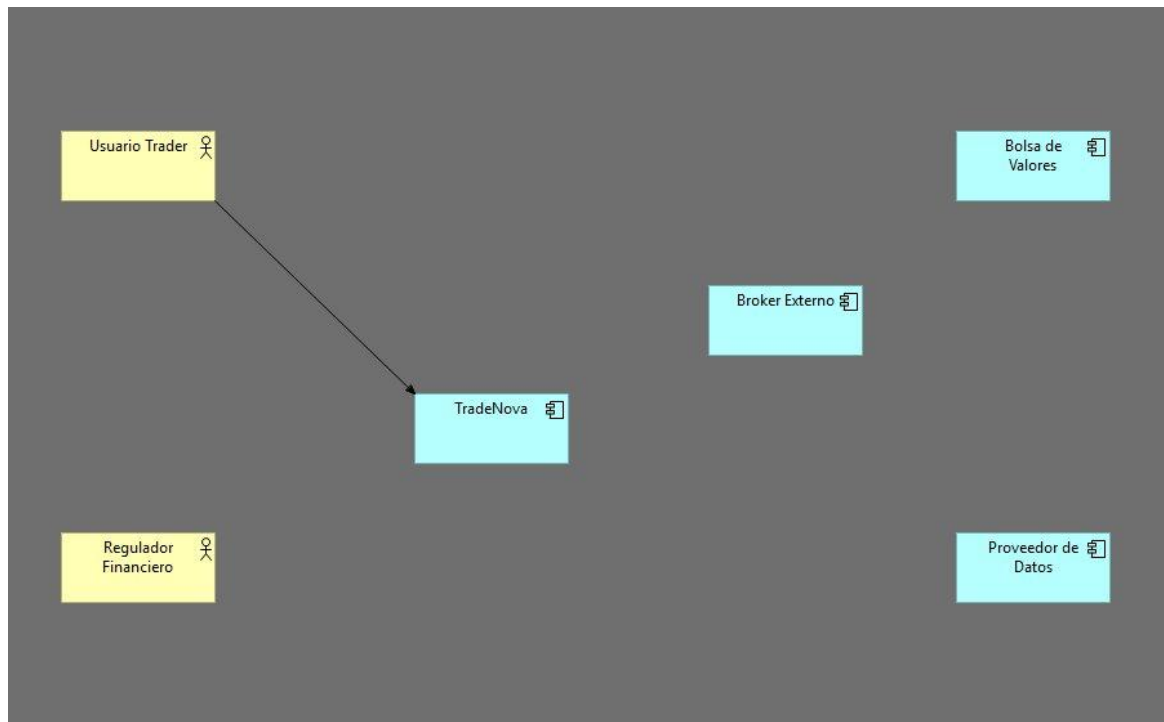


Figura 15. Primera conexión establecida — Usuario Trader hacia TradeNova

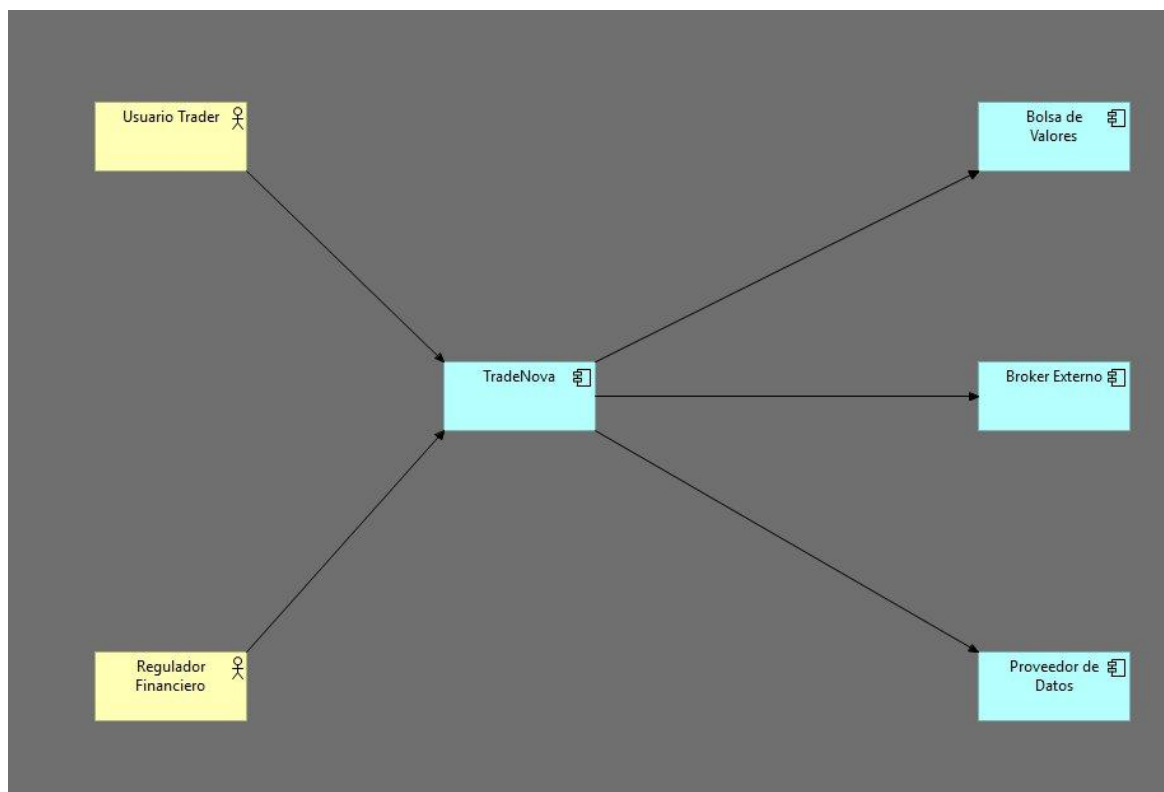


Figura 16. Todas las conexiones establecidas entre elementos del diagrama

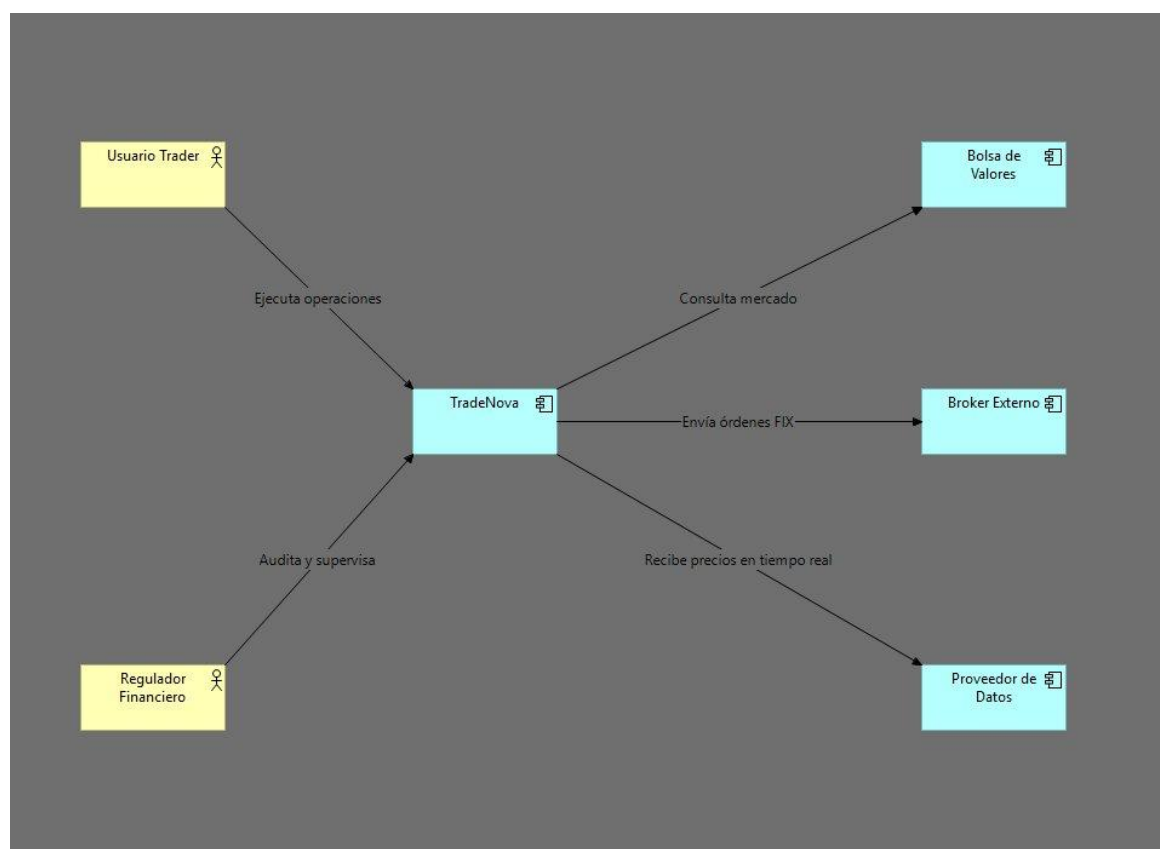


Figura 17. Diagrama de Contexto completo con etiquetas en las conexiones

Elementos del Diagrama

Elemento	Tipo ArchiMate	Descripción
TradeNova	Application Component	Plataforma de corretaje financiero — sistema central
Usuario Trader	Business Actor	Usuario minorista que ejecuta operaciones bursátiles
Regulator Financiero	Business Actor	Entidad regulatoria que audita y supervisa la plataforma
Broker Externo	Application Component	Ejecuta las órdenes financieras en los mercados
Bolsa de Valores	Application Component	NYSE, NASDAQ y otras bolsas de valores
Proveedor de Datos	Application Component	Proveedor de precios y cotizaciones en tiempo real

Diagrama Final

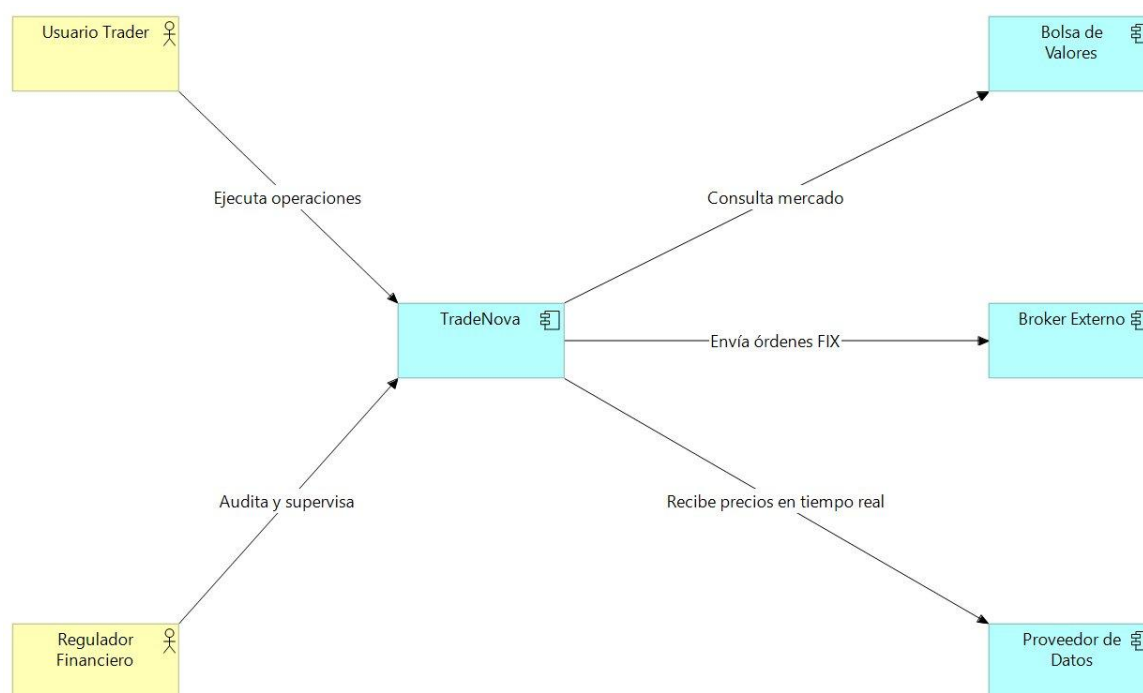


Figura 18. Diagrama de Contexto Final — TradeNova C4 (Nivel 1)

Análisis de SLAs, Dependencias y Constraints Regulatorios

Un análisis arquitectónico de nivel senior exige ir más allá de identificar los actores y sistemas externos: es necesario definir explícitamente los acuerdos de nivel de servicio (SLA) con cada dependencia externa, los riesgos de dependencia que estas representan y los constraints regulatorios que condicionan el diseño del sistema.

Actor / Sistema	SLA Esperado	Riesgo de Dependencia	Constraint Regulatorio
Broker Externo	Latencia FIX < 5 ms, disponibilidad 99.95%	Fallo del broker bloquea ejecución de órdenes. Se requiere failover a broker secundario.	MiFID II: trazabilidad completa de cada orden enviada con timestamp de nanosegundo
Bolsa de Valores	Feed de datos < 1 ms de delay, uptime 99.99% en horario bursátil	Datos desactualizados generan slippage y pérdidas financieras directas.	Regulación SEC/FINRA: prohibición de operar con datos de mercado con más de 15 min de delay
Proveedor de Datos	Latencia feed < 500 µs, disponibilidad 99.999%	Proveedor único genera SPOF crítico. Requiere proveedor secundario	Datos deben ser certificados y auditables. Proveedor debe estar registrado ante regulador.

Regulador Financiero	Respuesta a auditorías < 24h, reportes automáticos diarios	(redundancia activo-pasivo). Incumplimiento regulatorio genera multas y suspensión de operaciones.	SOX, MiFID II, Basilea III: reporte de operaciones sospechosas, capital mínimo, segregación de fondos
Usuario Trader	Latencia percibida < 200 ms end-to-end, disponibilidad 99.99%	Experiencia degradada en picos genera abandono y pérdida de confianza.	KYC/AML obligatorio: verificación de identidad, prevención de lavado de activos

Trade-offs Arquitectónicos del Contexto

A nivel de contexto, la arquitectura de TradeNova enfrenta trade-offs fundamentales que condicionan todo el diseño posterior:

Trade-off	Decisión	Consecuencia
Latencia vs. Consistencia	Priorizar latencia < 10 ms sobre consistencia fuerte	Se acepta consistencia eventual en lecturas no críticas; las órdenes son ACID.
Multi-broker vs. Single broker	Integración con múltiples brokers con enrutamiento inteligente	Mayor complejidad operativa a cambio de resiliencia y mejor precio de ejecución.
Cloud único vs. Multi-region	Despliegue multi-region activo-activo en AWS us-east-1 y eu-west-1	Mayor costo operativo, pero garantiza RPO < 1 min y RTO < 5 min ante desastre regional.
Datos en tiempo real vs. Batch	Streaming en tiempo real para precios y órdenes	Complejidad de infraestructura (Kafka, Flink) a cambio de latencia sub-segundo.

4.2 Diagrama de Contenedores (Nivel 2)

Descripción

El Diagrama de Contenedores descompone el sistema TradeNova en sus grandes piezas tecnológicas que pueden ejecutarse de manera independiente y comunicarse entre sí. En este nivel se evidencia la transición desde el monolito hacia una arquitectura de microservicios donde cada contenedor es responsable de un dominio funcional específico.

Proceso de construcción en Archi

Se creó la vista '02 - Diagrama de Contenedores' y se agregó un Application Component de gran tamaño representando el sistema TradeNova completo como contenedor padre. Dentro de él se arrastraron 6 Application Components adicionales, cada uno representando un microservicio o pieza tecnológica independiente. Al soltar cada componente dentro del contenedor padre, Archi presentó el diálogo de relación anidada, seleccionando Composition relation para indicar que cada pieza es parte integral del sistema.

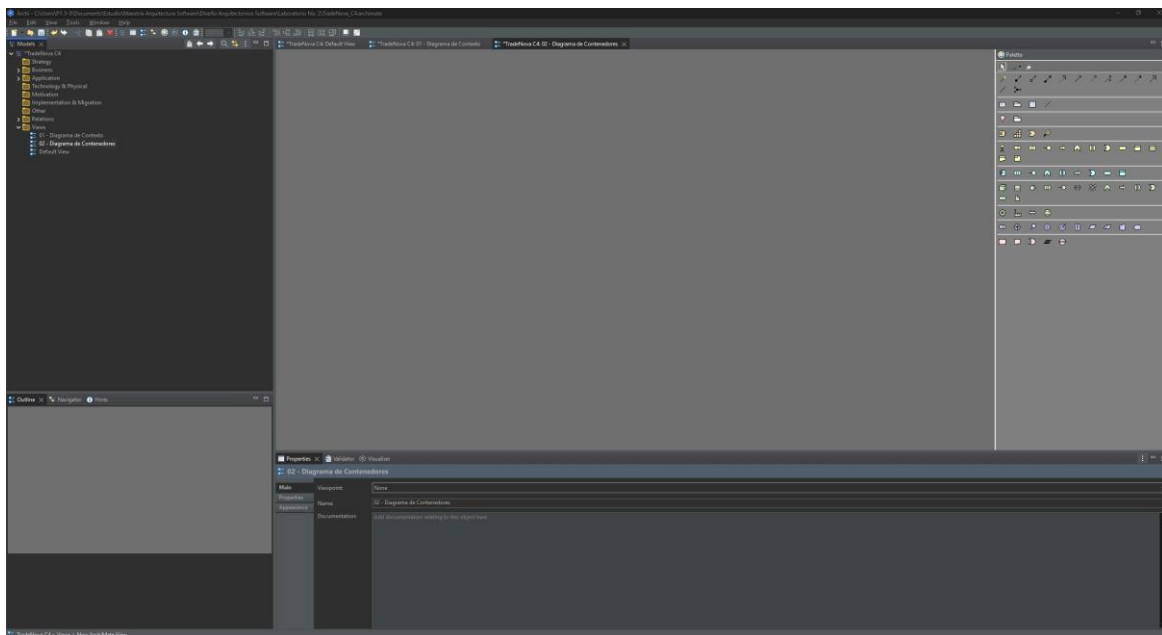


Figura 19. Vista 02 - Diagrama de Contenedores creada en Archi

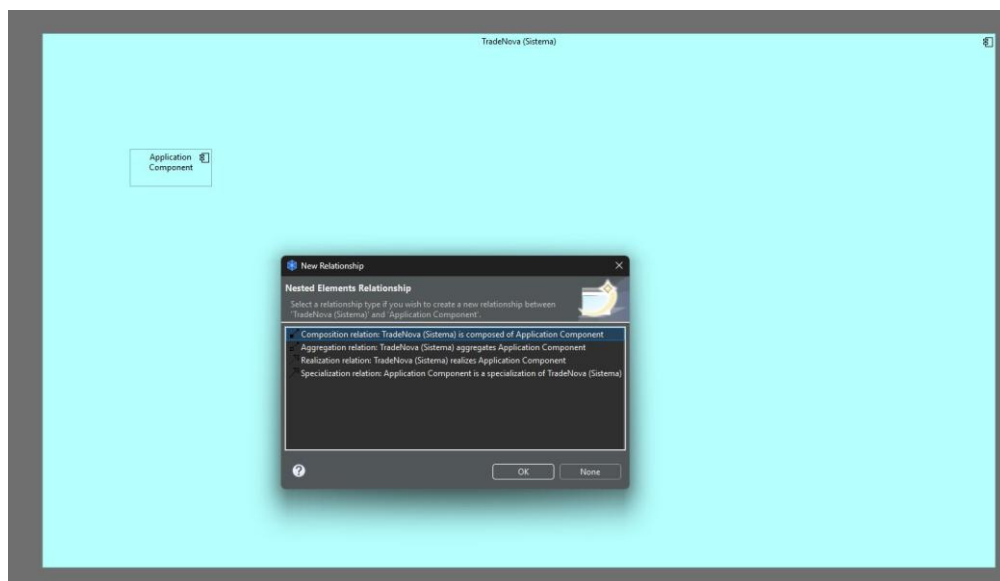


Figura 20. Diálogo de Composition relation al anidar componentes dentro del contenedor padre

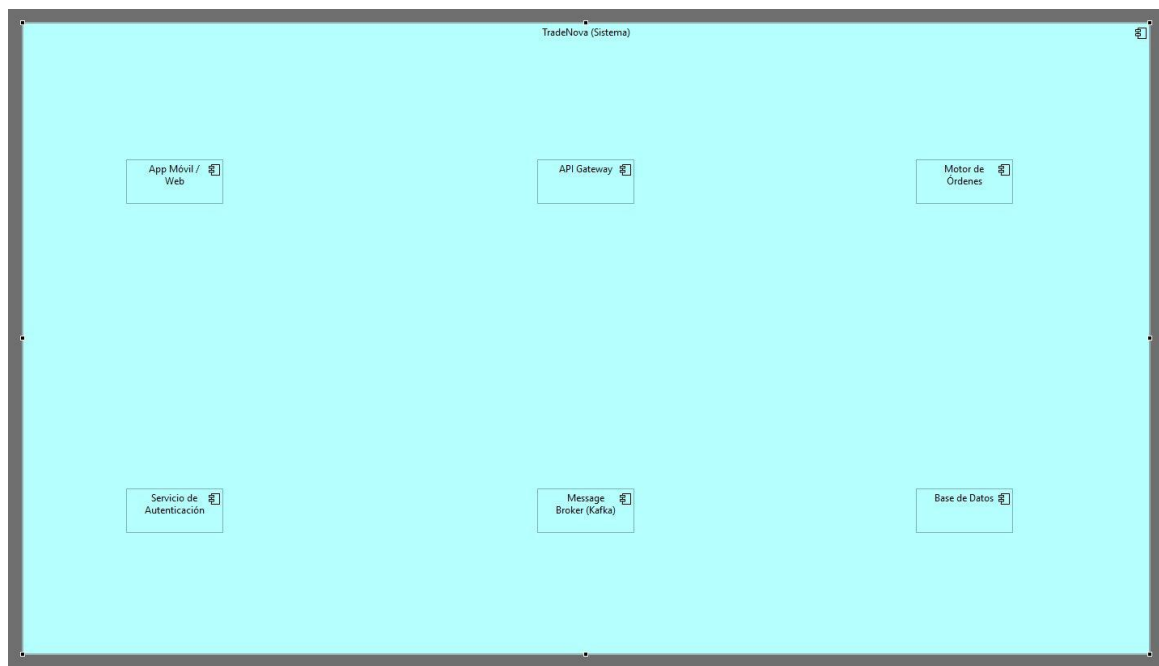


Figura 21. Los 6 contenedores tecnológicos dentro de TradeNova (Sistema)

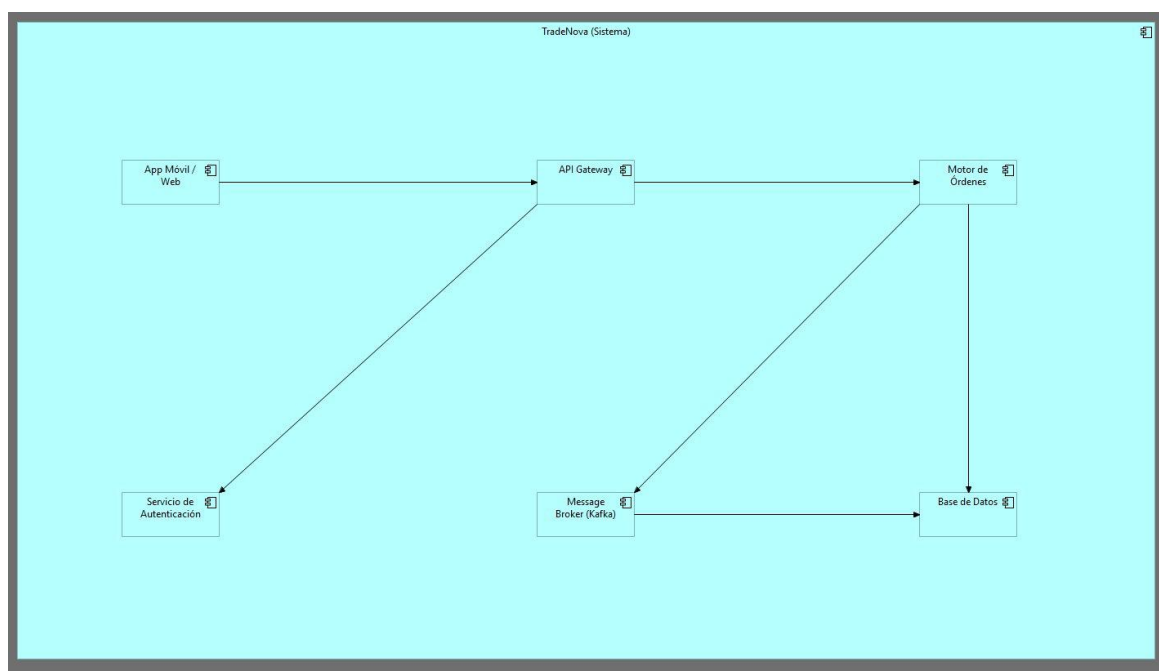


Figura 22. Diagrama de Contenedores con todas las conexiones establecidas

Elementos del Diagrama

Contenedor	Tecnología	Responsabilidad
App Móvil / Web	React / React Native	Interfaz de usuario para ejecución de operaciones
API Gateway	Kong / AWS API Gateway	Punto único de entrada, enrutamiento y rate limiting

Motor de Órdenes	Java / Spring Boot	Procesamiento transaccional en tiempo real
Servicio de Autenticación	Keycloak / OAuth2	Gestión de identidades y MFA
Message Broker (Kafka)	Apache Kafka	Cola de eventos para comunicación asíncrona
Base de Datos	PostgreSQL / Redis	Persistencia transaccional y caché

Diagrama Final

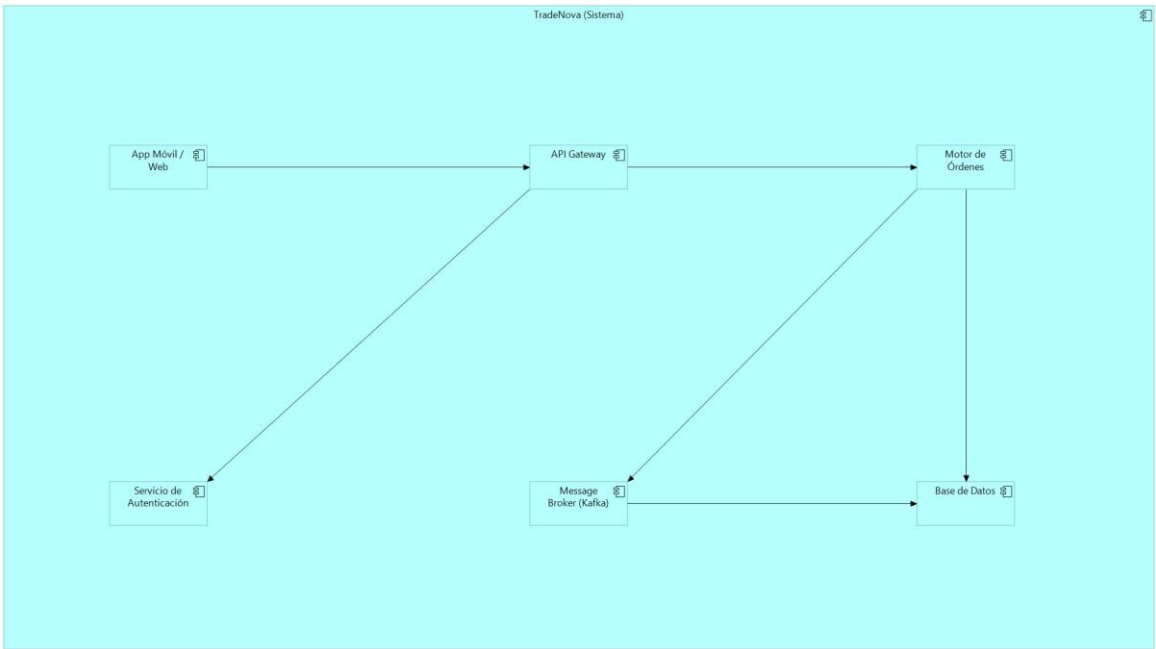


Figura 23. Diagrama de Contenedores Final — TradeNova C4 (Nivel 2)

Estrategia de Deployment — Kubernetes y Multi-Region

En un sistema financiero de misión crítica como TradeNova, la estrategia de deployment es una decisión arquitectónica de primer nivel. La plataforma se despliega sobre Kubernetes (EKS en AWS) en configuración multi-region activo-activo, con los siguientes principios:

Contenedor	Estrategia de Deployment	Réplicas Mínimas	Región
App Móvil / Web	CDN + S3 static hosting con CloudFront	N/A (serverless)	Global (edge locations)
API Gateway	Kong en Kubernetes con HPA activado	3 pods por región	us-east-1 + eu-west-1

Motor de Órdenes	Deployment K8s con anti-affinity rules (pods en nodos distintos)	5 pods por región	us-east-1 + eu-west-1
Servicio de Autenticación	Keycloak cluster con sesiones replicadas	3 pods por región	us-east-1 + eu-west-1
Message Broker (Kafka)	MSK Managed Kafka con replicación cross-region	3 brokers por región	us-east-1 + eu-west-1
Base de Datos	Aurora PostgreSQL Multi-AZ + Redis ElastiCache cluster	Multi-AZ automático	us-east-1 primary

Observabilidad — Logs, Tracing y Metrics

Sin observabilidad, un sistema financiero en producción es inoperable. TradeNova implementa la tríada de observabilidad completa (logs estructurados, distributed tracing y métricas en tiempo real) como requisito arquitectónico no negociable:

Dimensión	Herramienta	Qué monitorea	Alerta Crítica
Logs estructurados	ELK Stack (Elasticsearch + Logstash + Kibana)	Cada transacción, error y evento del sistema con correlationId	Error rate > 0.1% en órdenes genera alerta PagerDuty inmediata
Distributed Tracing	AWS X-Ray + OpenTelemetry	Latencia end-to-end por cada orden e identificación de cuellos de botella	P99 latencia > 10 ms activa escala automática del Motor de Órdenes
Métricas	Prometheus + Grafana	CPU, memoria, throughput de órdenes, profundidad de cola Kafka, error rates	Cola Kafka > 10.000 mensajes pendientes genera alerta de capacidad
APM	Datadog APM	Performance de cada método crítico del Motor de Órdenes	GC pauses > 5 ms activan análisis de heap inmediato
Alerting	PagerDuty + Opsgenie	On-call rotation 24/7 para incidentes P1 y P2	Downtime > 30 seg escala automáticamente a nivel CTO

Manejo de Fallos Distribuidos — Resiliencia

En una arquitectura de microservicios financiera, los fallos en cascada pueden resultar en pérdidas millonarias. TradeNova implementa los siguientes patrones de resiliencia en todos sus contenedores:

Patrón	Implementación	Contenedor Aplicado	Comportamiento ante Fallo
Circuit Breaker	Resilience4j con umbrales configurables	Motor de Órdenes hacia Broker Externo	Si falla rate > 50% en 10s: abre circuito, retorna error controlado, reintenta cada 30s
Retry with Backoff	Exponential backoff: 100ms, 200ms, 400ms máx 3 reintentos	API Gateway y Motor de Órdenes	Reintentos automáticos ante errores transitorios de red o timeout
Bulkhead	Thread pool isolation por operación crítica	Motor de Órdenes	Pool de validación separado del pool de enrutamiento FIX. Un cuello de botella no bloquea al otro.
Timeout	Timeouts agresivos: DB 50ms, Broker 100ms, Externo 500ms	Todos los contenedores	Evita que esperas indefinidas agoten recursos del sistema
Fallback	Respuesta degradada controlada	API Gateway	Si Servicio de Autenticación no responde en 200ms: rechaza con HTTP 503 claro al usuario

4.3 Diagrama de Componentes (Nivel 3)

Descripción

El Diagrama de Componentes descompone el contenedor Motor de Órdenes en sus componentes internos. Este contenedor fue seleccionado por ser el más crítico de la plataforma: garantiza la latencia ultra-baja (< 10 ms), aplica las reglas de riesgo que previenen errores tipo Fat Finger y enruta las órdenes mediante el protocolo FIX.

Proceso de construcción en Archi

Se creó la vista '03 - Diagrama de Componentes' y se construyó de manera análoga al diagrama anterior: un Application Component grande representando el Motor de Órdenes como contenedor, y 5 Application Components internos representando cada componente funcional, todos conectados mediante relaciones de Composition y Association.

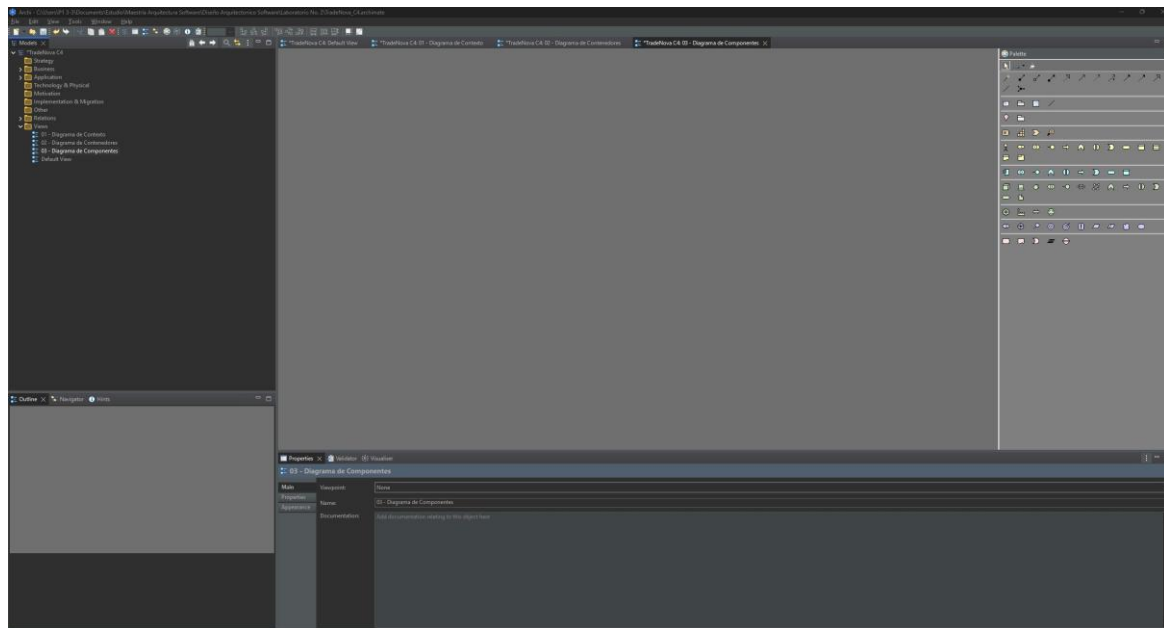


Figura 24. Vista 03 - Diagrama de Componentes creada en Archi

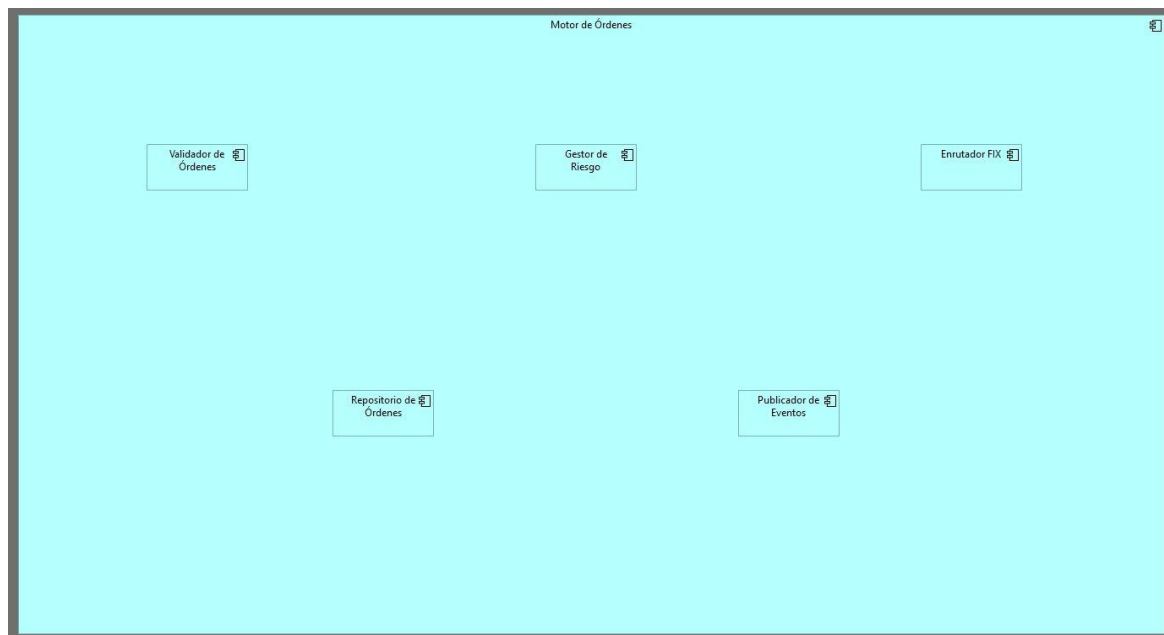


Figura 25. Los 5 componentes del Motor de Órdenes agregados al canvas

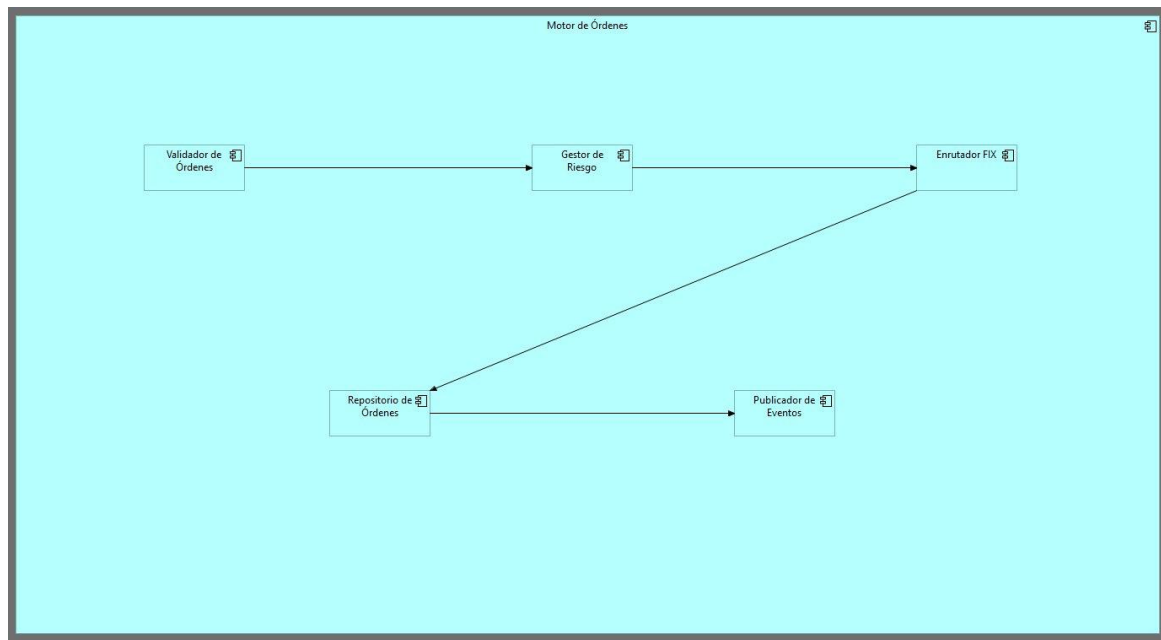


Figura 26. Diagrama de Componentes con el pipeline de procesamiento conectado

Elementos del Diagrama

Componente	Responsabilidad
Validador de Órdenes	Orquesta la validación: fondos, formato y reglas de riesgo
Gestor de Riesgo	Detecta Fat Finger, límites de exposición y restricciones regulatorias
Enrutador FIX	Traduce y envía órdenes al broker externo via protocolo FIX
Repositorio de Órdenes	Persiste el estado de cada orden con trazabilidad completa
Publicador de Eventos	Publica eventos al Message Broker (Kafka)

Diagrama Final

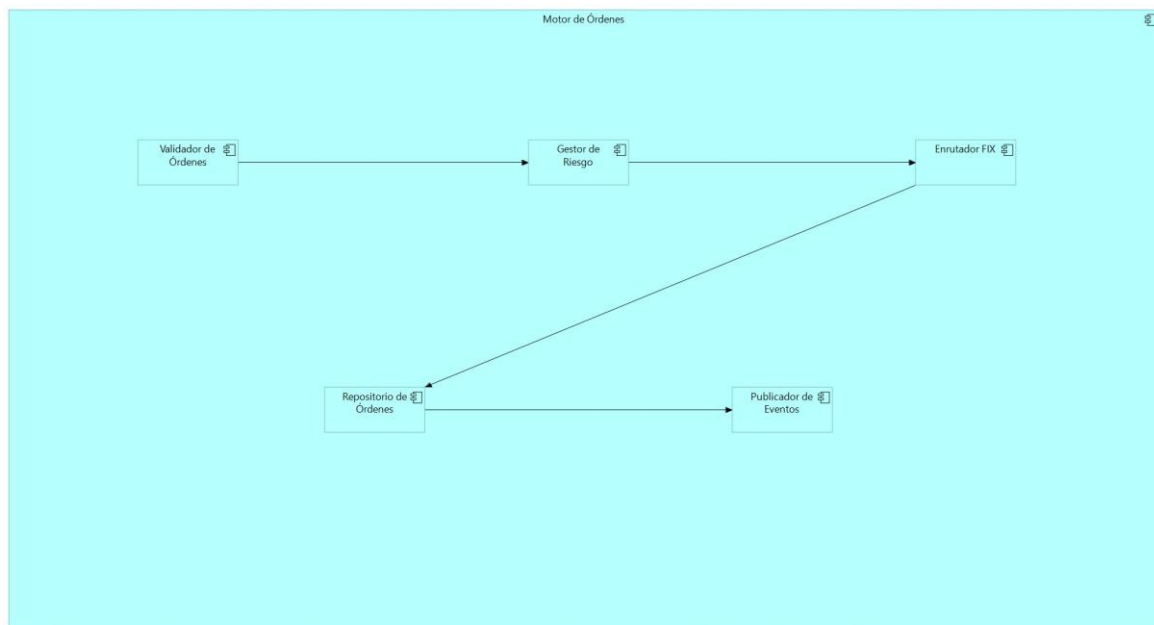


Figura 27. Diagrama de Componentes Final — Motor de Órdenes, TradeNova C4 (Nivel 3)

Pipeline Reactivo — Paralelismo, Backpressure y Retry Strategies

Un arquitecto senior no diseña el Motor de Órdenes como un pipeline puramente secuencial. En producción financiera de alta concurrencia, el pipeline debe ser reactivo y orientado a eventos para evitar cuellos de botella y garantizar la latencia objetivo bajo carga extrema. A continuación se describe el diseño de nivel senior para cada aspecto crítico:

Aspecto	Problema en Pipeline Secuencial	Solución Senior	Implementación
Paralelismo	Validación de fondos y reglas de riesgo ejecutadas en serie suma latencias	Ejecución paralela con <code>CompletableFuture.allOf()</code>	<code>ValidadorFondos</code> y <code>ValidadorReglas</code> corren en paralelo; <code>GestorRiesgo</code> recibe ambos resultados antes de decidir
Backpressure	Picos de demanda colapsan la cola interna del Motor de Órdenes	Reactive Streams con Project Reactor (Flux/Mono) y buffer con overflow strategy <code>DROP_LATEST</code>	Si el sistema está saturado, rechaza nuevas órdenes con HTTP 429 y mensaje claro al usuario
Retry Strategy	Fallos transitorios en <code>EnrutadorFIX</code> pierden órdenes definitivamente	Dead Letter Queue (DLQ) en Kafka + retry automático con backoff exponencial	Órdenes fallidas van a topic <code>kafka:orders.dlq</code> ; worker dedicado

Idempotencia	Reintentos duplican órdenes en el mercado generando pérdidas	Idempotency key por orden (UUID v4) validado antes de procesar	reintenta cada 5min hasta 3 veces Si la orden ya existe en Redis (TTL 5min): retorna resultado previo sin re-ejecutar
Event-Driven Flow	Pipeline acoplado impide evolución independiente de componentes	Cada componente publica/consume eventos de Kafka internamente	Validador publica OrdenValidada, GestorRiesgo consume y publica OrdenAprobada, EnrutadorFIX consume

Seguridad Avanzada en el Motor de Órdenes

Más allá de OAuth2, el Motor de Órdenes implementa controles de seguridad específicos del dominio financiero:

Control	Descripción	Regulación Aplicable
Zero Trust interno	Cada componente verifica el JWT del llamador incluso dentro del mismo pod. No existe comunicación sin autenticación.	MiFID II, SOX
Encriptación en tránsito	mTLS entre todos los componentes internos del Motor de Órdenes	PCI-DSS, GDPR
Encriptación en reposo	AES-256 para todos los datos de órdenes en PostgreSQL y Redis	SOX, regulación financiera local
Auditoría financiera	Cada decisión del GestorRiesgo genera un registro inmutable en append-only log (Event Sourcing)	MiFID II: obligación de conservar audit trail 7 años
Prevención de manipulación	Firma HMAC de cada mensaje entre componentes para detectar tampering	Regulación anti-fraude SEC

4.4 Diagrama de Código (Nivel 4)

Descripción

El Diagrama de Código expone la estructura interna del componente Validador de Órdenes mostrando las clases que lo implementan. Este componente fue seleccionado por ser el punto de entrada del pipeline de procesamiento y por implementar la lógica de validación más crítica del sistema.

Proceso de construcción en Archi

Se creó la vista '04 - Diagrama de Código' siguiendo el mismo patrón de construcción: un Application Component contenedor representando el Validador de Órdenes y 4 Application Components internos representando las clases del componente.

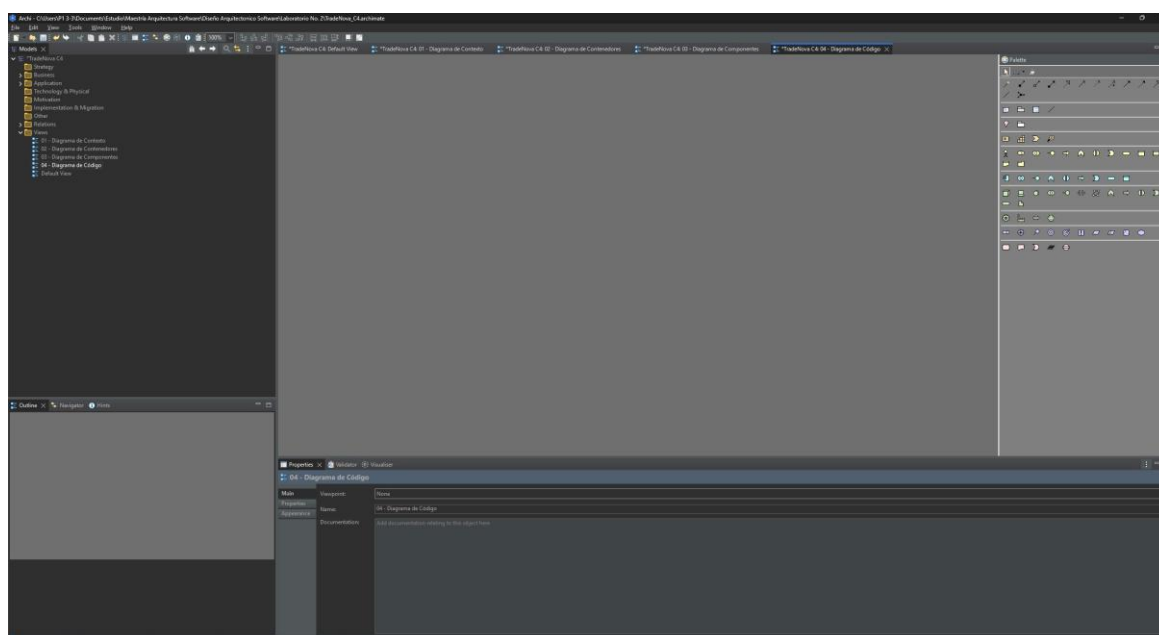


Figura 28. Vista 04 - Diagrama de Código creada en Archi

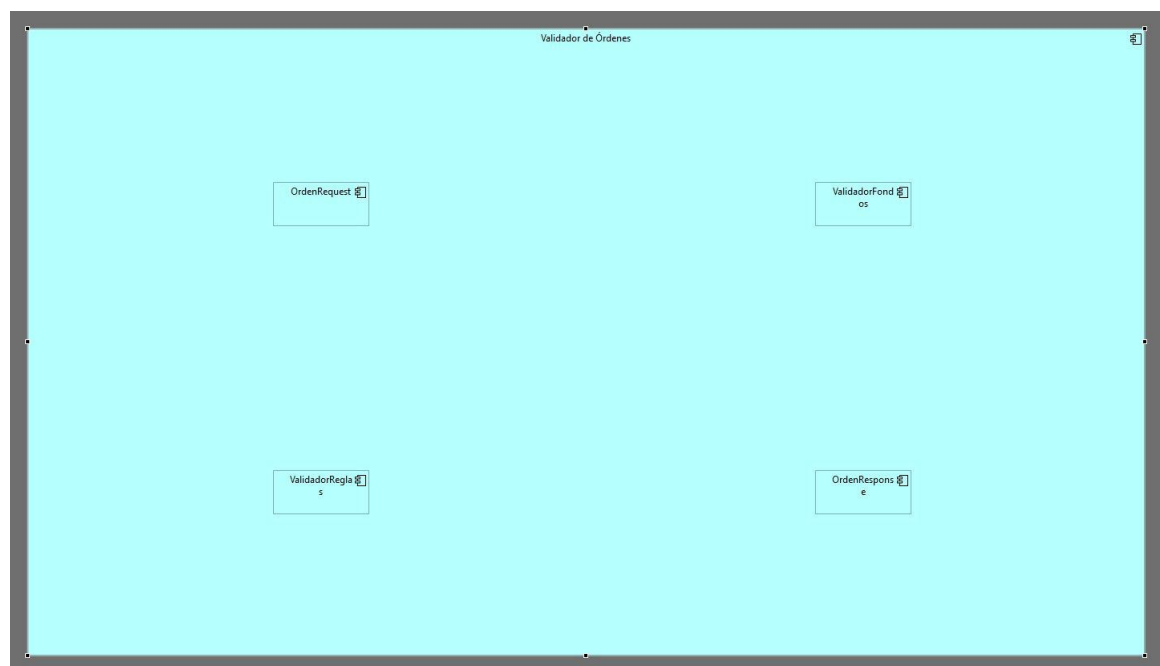


Figura 29. Las 4 clases del Validador de Órdenes agregadas al canvas

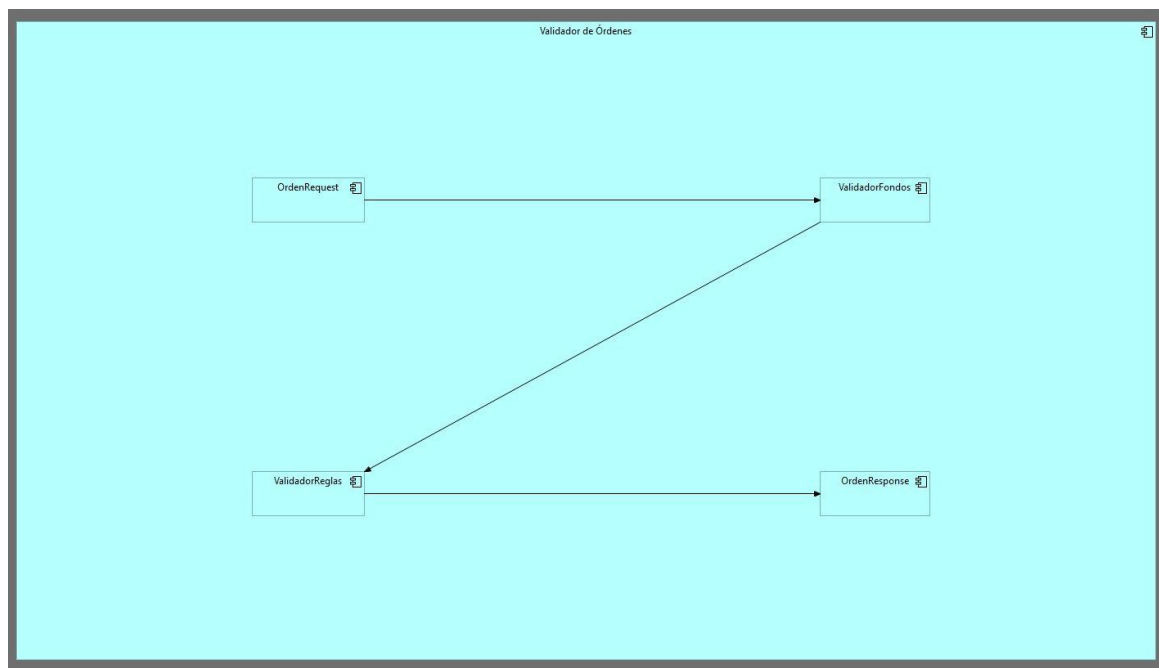


Figura 30. Diagrama de Código con las relaciones entre clases establecidas

Elementos del Diagrama

Clase	Tipo	Responsabilidad
OrdenRequest	DTO entrada	Encapsula los datos de la orden entrante
ValidadorFondos	Service Component	Verifica saldo y bloquea monto requerido
ValidadorReglas	Service Component	Aplica reglas de negocio y detecta anomalías
OrdenResponse	DTO salida	Encapsula el resultado de la validación

Diagrama Final

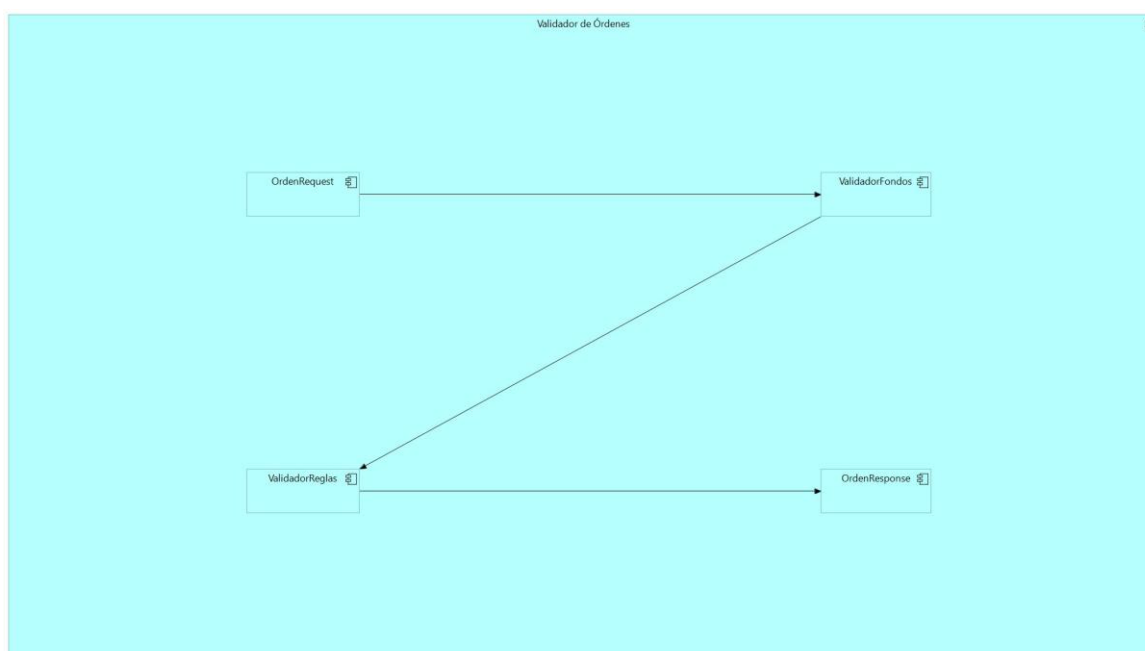


Figura 31. Diagrama de Código Final — Validador de Órdenes, TradeNova C4 (Nivel 4)

Patrones de Diseño Aplicados

Un Validador de Órdenes de nivel senior no es simplemente un conjunto de if/else. Aplica patrones de diseño que garantizan extensibilidad, testabilidad y cumplimiento del principio Open/Closed (abierto para extensión, cerrado para modificación):

Patrón	Aplicación en el Validador	Beneficio
Strategy	Cada regla de validación (fondos, límites, Fat Finger, regulatorio) es una implementación de la interfaz IValidationStrategy	Agregar nuevas reglas no modifica código existente — solo se registra una nueva Strategy en el contexto
Chain of Responsibility	Las validaciones se encadenan: cada nodo decide si pasa o detiene el flujo y enriquece el contexto con su resultado	El orden de validación es configurable en tiempo de ejecución sin recompilar
Builder	OrdenRequest se construye con un Builder que valida campos obligatorios en construcción	Imposible crear una OrdenRequest inválida — la validación de estructura ocurre antes del pipeline
Decorator	ValidadorFondos puede ser decorado con caché (Redis) para reutilizar consultas de saldo en ventana de 100ms	Reduce latencia y carga en base de datos durante picos de demanda
Null Object	OrdenResponse implementa un	Elimina null checks en capas superiores —

	NullOrdenResponse para respuestas de rechazo estructuradas	siempre hay una respuesta válida
--	--	----------------------------------

Manejo de Errores Estructurado

El manejo de errores en un sistema financiero no puede ser ad-hoc. TradeNova implementa una jerarquía de excepciones de dominio y una estrategia de respuesta consistente en todos los componentes del Validador:

Tipo de Error	Excepción de Dominio	Acción	Respuesta al Usuario
Fondos insuficientes	InsufficientFundsException	Registra en audit log, libera bloqueo preventivo	HTTP 422 con código TN-4001 y monto faltante
Fat Finger detectado	AnomalousOrderException	Alerta al equipo de riesgo, suspende orden 60s para revisión manual	HTTP 422 con código TN-4002 y solicitud de confirmación explícita
Violación regulatoria	RegulatoryViolationException	Bloqueo inmediato, notificación al compliance officer, registro en audit trail regulatorio	HTTP 403 con código TN-4003 — operación permanentemente bloqueada
Timeout de validación	ValidationTimeoutException	Circuit breaker se activa, orden rechazada de forma segura	HTTP 503 con código TN-5001 y tiempo estimado de recuperación
Error de sistema	SystemException	Dead letter queue, alerta P1, rollback de bloqueo de fondos	HTTP 500 con correlationId para rastreo en logs

Consistencia de Datos — ACID vs. Eventual Consistency

Una de las decisiones más críticas en la arquitectura de TradeNova es la estrategia de consistencia de datos. No todos los datos requieren el mismo nivel de garantía, y una aplicación uniforme de ACID penalizaría la latencia. La estrategia adoptada es selectiva:

Operación	Estrategia	Justificación
-----------	------------	---------------

Ejecución de órdenes	ACID estricto (PostgreSQL transacciones serializables)	Una orden ejecutada dos veces o a precio incorrecto genera pérdida financiera directa. No se acepta ningún grado de inconsistencia.
Bloqueo de fondos	ACID con optimistic locking	El bloqueo debe ser atómico para evitar race conditions en órdenes concurrentes del mismo usuario.
Eventos a Kafka	Exactly-once semantics (EOS) con transacciones Kafka	Exactly-once garantiza que cada evento de orden se procesa exactamente una vez en todos los consumidores. Se configura con <code>enable.idempotence=true</code> y <code>transactional.id</code> .
Datos de mercado (precios)	Eventual consistency aceptada	Un precio con 50ms de delay es aceptable para visualización. La ejecución siempre usa el precio del momento de la orden.
Historial de órdenes	Eventual consistency con Event Sourcing	El estado final se reconstruye desde los eventos. Puede haber delay de segundos en reportes, pero el audit trail es inmutable.

6. Consideraciones de Arquitectura Global

Una arquitectura de nivel senior no solo diseña los cuatro niveles C4 — también define las preocupaciones transversales que determinan si el sistema puede operar de manera segura, sostenible y evolutiva en un entorno financiero de producción real. Esta sección consolida las decisiones arquitectónicas globales de TradeNova que trascienden cualquier nivel individual del Modelo C4.

6.1 Seguridad — Zero Trust Architecture

TradeNova adopta el modelo Zero Trust: ningún componente, usuario o red es confiable por defecto, independientemente de si está dentro o fuera del perímetro del sistema. Cada solicitud debe ser autenticada, autorizada y cifrada en todo momento.

Principio Zero Trust	Implementación en TradeNova	Herramienta
----------------------	-----------------------------	-------------

Verificar explícitamente	Cada microservicio verifica el JWT en cada request, incluso en comunicación interna entre pods	Keycloak + Spring Security
Mínimo privilegio	Cada microservicio tiene una service account con permisos mínimos necesarios (RBAC estricto)	Kubernetes RBAC + AWS IAM Roles
Asumir breach	Segmentación de red con Network Policies en K8s. El Motor de Órdenes solo acepta tráfico del API Gateway.	Kubernetes Network Policies + AWS Security Groups
mTLS everywhere	Comunicación cifrada con TLS mutuo entre todos los microservicios. Certificados rotados automáticamente.	Istio Service Mesh + cert-manager
Encriptación end-to-end	Datos cifrados en tránsito (TLS 1.3) y en reposo (AES-256). Claves gestionadas en AWS KMS.	AWS KMS + TLS 1.3
Auditoría continua	Cada acción de usuario y sistema genera un registro inmutable en audit trail centralizado	CloudTrail + ELK Stack

6.2 DevOps — CI/CD e Infraestructura como Código

En un sistema financiero en producción, los despliegues manuales son inaceptables. TradeNova implementa una pipeline de CI/CD completamente automatizada con gates de calidad obligatorios antes de cualquier despliegue a producción:

Etapas CI/CD	Herramienta	Criterio de Paso	Tiempo Máximo
Source Control	GitHub con branch protection rules	PR obligatorio con 2 aprobaciones + passing checks	N/A
Build & Unit Tests	GitHub Actions + Maven/Gradle	Cobertura de tests > 80%, 0 tests fallidos	5 minutos
Static Analysis	SonarQube + Checkmarx (SAST)	0 vulnerabilidades críticas, 0 code smells bloqueantes	8 minutos
Container Security Scan	Trivy + AWS ECR scanning	0 vulnerabilidades CRITICAL en imagen Docker	3 minutos

Integration Tests	Testcontainers + Docker Compose	100% tests de integración passing	10 minutos
Performance Tests	k6 con SLA gates	P99 < 10ms, throughput > 10.000 órdenes/seg	15 minutos
Deploy Staging	ArgoCD (GitOps) en EKS staging	Smoke tests automáticos passing	5 minutos
Deploy Production	ArgoCD con canary deployment (5% → 25% → 100%)	Error rate < 0.01% en cada fase antes de avanzar	30 minutos

La infraestructura completa de TradeNova está definida como código (IaC) usando Terraform para recursos AWS y Helm Charts para configuración de Kubernetes. Esto garantiza que cualquier entorno (dev, staging, producción) pueda ser recreado en minutos de forma determinista, eliminando la deuda de configuración manual.

6.3 Consistencia y Gestión de Eventos en Kafka

La configuración de Apache Kafka en TradeNova sigue principios de producción financiera con exactly-once semantics para garantizar que ninguna orden se procese más de una vez ni se pierda ante fallos:

Configuración Kafka	Valor	Justificación
enable.idempotence	true	El productor asigna un sequence number a cada mensaje. El broker rechaza duplicados automáticamente.
acks	all	El líder espera confirmación de todas las réplicas antes de confirmar escritura. Sin pérdida de datos ante fallo del líder.
transactional.id	order-engine-{instanceId}	Permite transacciones atómicas: publicar a múltiples topics o no publicar nada (atomicidad distribuida).
min.insync.replicas	2	Mínimo 2 réplicas en sync antes de aceptar escritura. Factor de replicación = 3.
isolation.level (consumidor)	read_committed	El consumidor solo lee mensajes de transacciones

retention.ms	604800000 (7 días)	completadas. Elimina lecturas sucias. Mensajes retenidos 7 días para replay ante incidentes o auditorías regulatorias.
--------------	--------------------	---

6.4 Estrategia de Recuperación ante Desastres

TradeNova define objetivos de recuperación específicos y los mecanismos arquitectónicos que los garantizan:

Métrica	Objetivo	Mecanismo	Validación
RPO (Recovery Point Objective)	< 1 minuto de datos perdidos ante desastre regional	Replicación continua de Aurora PostgreSQL cross-region + Kafka MirrorMaker 2 cross-region	Simulacro trimestral de failover
RTO (Recovery Time Objective)	< 5 minutos para restauración completa del servicio	Route53 health checks + failover automático a región secundaria + pods pre-calentados en standby	Simulacro trimestral de failover
MTTR (Mean Time To Recover)	< 15 minutos para incidentes P1	Runbooks automatizados + PagerDuty on-call + ArgoCD rollback en 1 click	Post-mortems mensuales
Backup de datos	Snapshots diarios + continuous backup	AWS Backup + Aurora PITR (Point-In-Time Recovery) hasta 35 días	Restauración de prueba mensual

7. Exportación de los Diagramas

Una vez finalizados los cuatro diagramas C4, se procedió a exportarlos como imágenes PNG desde Archi. El proceso de exportación se realizó para cada vista de manera individual siguiendo la ruta: File → Export → View As Image, configurando el formato PNG con escala al 150% para garantizar alta calidad visual en el documento final.

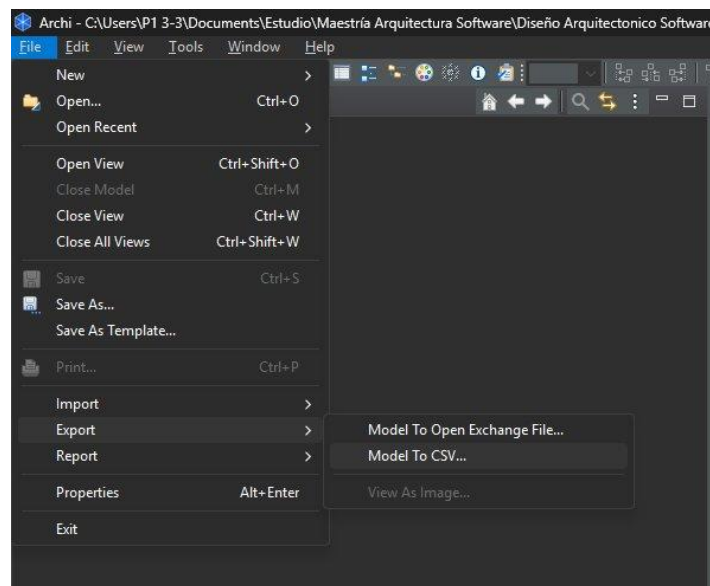


Figura 32. Menú de exportación File → Export en Archi

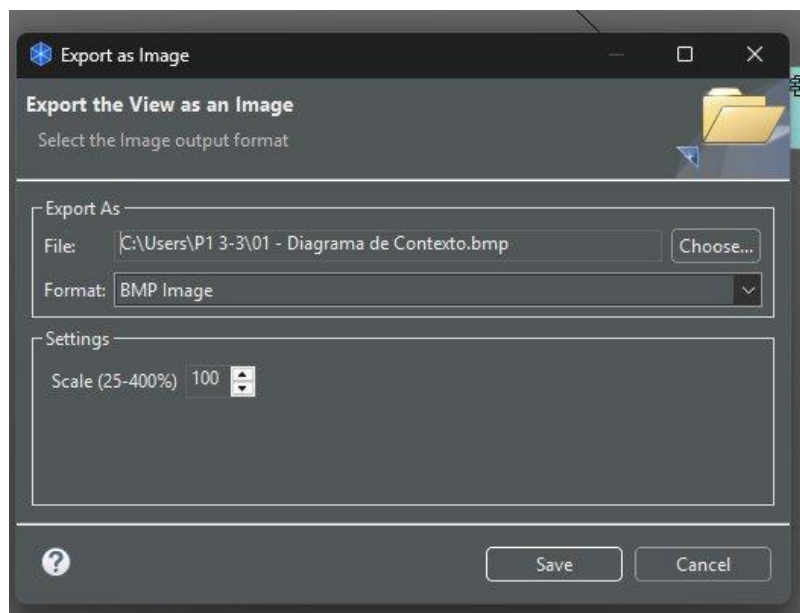


Figura 33. Diálogo de exportación — configuración de formato PNG y escala

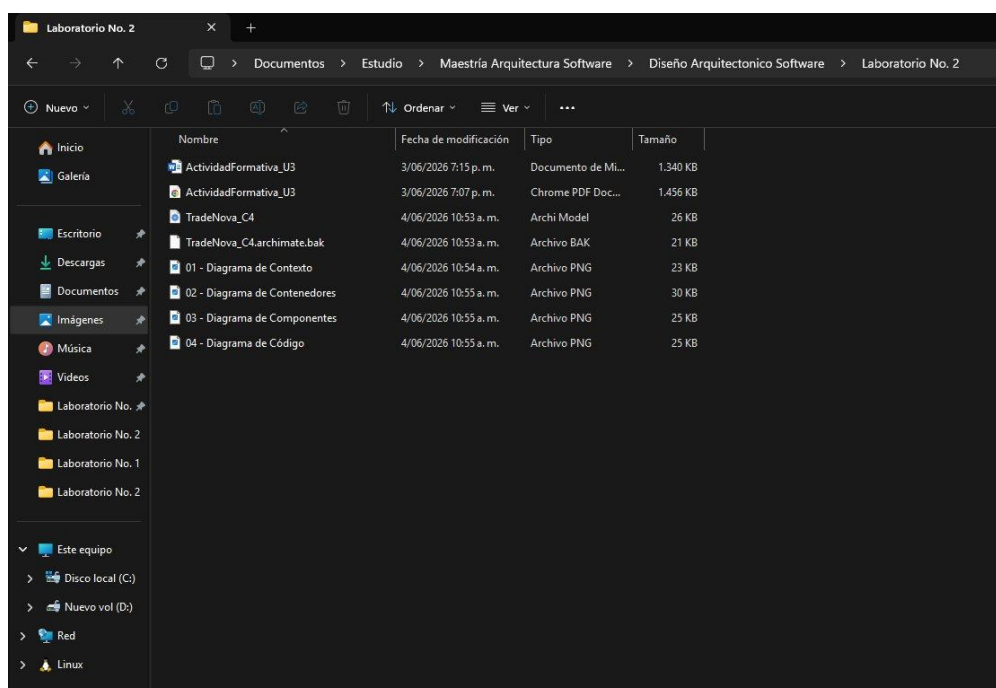


Figura 34. Los 4 diagramas exportados como PNG en la carpeta del Laboratorio No. 2

8. Conclusiones

La aplicación del Modelo C4 al diseño arquitectónico de TradeNova permitió documentar de manera estructurada, progresiva y comprensible la arquitectura de una plataforma financiera de misión crítica. A través de los cuatro niveles de abstracción, fue posible comunicar tanto la visión estratégica del sistema como el detalle técnico de sus componentes más sensibles, respondiendo a las necesidades de información de distintas audiencias involucradas en el proyecto.

El Diagrama de Contexto evidenció que TradeNova opera en un ecosistema financiero complejo, con múltiples actores externos que imponen requisitos estrictos de integración, seguridad e interoperabilidad. La relación con brokers externos mediante el protocolo FIX, la supervisión regulatoria continua y la dependencia de proveedores de datos en tiempo real representan factores arquitectónicos que condicionan directamente las decisiones de diseño del sistema.

El Diagrama de Contenedores confirmó que la transición hacia una arquitectura de microservicios es la respuesta adecuada a los desafíos de escalabilidad, disponibilidad y modificabilidad identificados en el análisis de atributos de calidad de la Unidad 2. La separación de responsabilidades entre el API Gateway, el Motor de Órdenes, el Servicio de Autenticación, el Message Broker y la Base de Datos permite que cada pieza evolucione, escale y falle de manera independiente.

El Diagrama de Componentes del Motor de Órdenes demostró cómo los atributos de calidad priorizados —especialmente latencia ultra-baja, tolerancia a fallos y usabilidad— se traducen en decisiones concretas de diseño: un pipeline de validación

secuencial y especializado donde cada componente tiene una responsabilidad bien definida, permitiendo ejecutar todo el procesamiento interno en menos de 10 milisegundos.

El Diagrama de Código del Validador de Órdenes evidenció cómo principios como la separación de responsabilidades (SRP) y el diseño orientado al dominio (DDD) se aplican en el nivel más granular de la arquitectura, produciendo clases cohesivas, testeables y fácilmente modificables.

Finalmente, el uso de Archi 5.9.0 como herramienta de modelado ArchiMate demostró ser una elección acertada para este tipo de ejercicios académicos y profesionales. Su interfaz intuitiva, la Palette organizada por capas y el soporte nativo para relaciones de composición y asociación permitieron construir los cuatro niveles del Modelo C4 de manera eficiente, produciendo diagramas claros y exportables en alta calidad para su inclusión en documentación técnica formal.

9. Referencias Bibliográficas

- Brown, S. (2018). The C4 Model for Software Architecture. <https://c4model.com>
- Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley.
- Richards, M., & Ford, N. (2020). Fundamentals of Software Architecture. O'Reilly Media.
- Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly Media.
- Kazman, R., Klein, M., & Clements, P. (2000). ATAM: Method for Architecture Evaluation. Software Engineering Institute.
- Barbacci, M., et al. (2003). Quality Attribute Workshops (QAWs), Third Edition. Software Engineering Institute.
- The Open Group. (2019). ArchiMate 3.1 Specification. <https://www.opengroup.org/archimate-forum>
- Archi Tool. (2026). Archi 5.9.0 User Guide. <https://www.archimatetool.com>
- Microsoft Azure Architecture Center. (2026). <https://docs.microsoft.com/azure/architecture>
- Amazon Web Services. (2026). AWS Architecture Center. <https://aws.amazon.com/architecture>
- FIX Trading Community. (2026). FIX Protocol Specification. <https://www.fixtrading.org>
- Material académico Diseño Arquitectónico de Software — Unidad 3 — Politécnico Grancolombiano — 2026.
- Clases virtuales y orientaciones metodológicas del docente John Rondón Suárez.